Namespace SixLabors.ImageSharp.Web

Classes

ApplicationBuilderExtensions
: Extension methods for IApplicationBuilder to simplify middleware registration.

CaseHandlingUriBuilder
: Optimized helper methods for generating encoded Uris from URI components. Much of this code has been adapted from the MIT licensed .NET runtime.

ExifOrientationUtilities
: Contains various helper methods for working with EXIF orientation values.

Format
: Contains reusable static instances of known image formats.

FormatUtilities
: Contains various helper methods for working with image formats based on the given configuration.

FormattedImage
: A class encapsulating an image with a particular file encoding.

HMACUtilities
: Provides methods to compute a Hash-based Message Authentication Code (HMAC).

ImageSharpBuilderExtensions
: Extension methods for IImageSharpBuilder that allow configuration of services.

RequestAuthorizationUtilities
: Contains various helper methods for authorizing image requests.

Resampler
: Contains reusable static instances of known resampling algorithms.

ServiceCollectionExtensions
: Extension methods for IServiceCollection to simplify middleware service registration.

Structs

FormatCommand
: Represents a command for setting the type of format with which to encode processed images.

ImageCacheMetadata
: Represents the metadata associated with an image file.

ImageMetadata
: Represents the metadata associated with an image file.

ResamplerCommand
: Represents a command for setting the type of sampler to use for resampling operations.

Interfaces

IImageSharpBuilder
: Defines a contract for adding ImageSharp services.

Enums

CaseHandlingUriBuilder.CaseHandling
: Provides Uri case handling options.

CommandHandling
: Provides enumeration for handling CommandCollection instances when processing a request.

Class ApplicationBuilderExtensions

Namespace: SixLabors.ImageSharp.Web

Assembly: SixLabors.ImageSharp.Web.dll

Extension methods for IApplicationBuilder to simplify middleware registration.

```
public static class ApplicationBuilderExtensions
```

Inheritance

object ← ApplicationBuilderExtensions

Inherited Members

object.Equals(object) , object.Equals(object, object) , object.GetHashCode() , object.GetType() , object.MemberwiseClone() , object.ReferenceEquals(object, object) , object.ToString()

Methods

UseImageSharp(IApplicationBuilder)

Registers the ImageSharp middleware.

```
public static IApplicationBuilder UseImageSharp(this IApplicationBuilder app)
```

Parameters

app IApplicationBuilder

The application with the mechanism to configure a request pipeline.

Returns

IApplicationBuilder

IApplicationBuilder.

Class CaseHandlingUriBuilder

Namespace: SixLabors.ImageSharp.Web

Assembly: SixLabors.ImageSharp.Web.dll

Optimized helper methods for generating encoded Uris from URI components. Much of this code has been adapted from the MIT licensed .NET runtime.

```
public static class CaseHandlingUriBuilder
```

Inheritance

object ← CaseHandlingUriBuilder

Inherited Members

object.Equals(object) , object.Equals(object, object) , object.GetHashCode() , object.GetType() , object.MemberwiseClone() , object.ReferenceEquals(object, object) , object.ToString()

Methods

BuildAbsolute(CaseHandling, HostString, PathString, PathString, QueryString)

Combines the given URI components into a string that is properly encoded for use in HTTP headers. Note that unicode in the HostString will be encoded as punycode and the scheme is not included in the result.

```
public static string BuildAbsolute(CaseHandlingUriBuilder.CaseHandling handling, HostString
host, PathString pathBase = default, PathString path = default, QueryString query = default)
```

Parameters

`handling` CaseHandlingUriBuilder.CaseHandling

Determines case handling for the result. `query` is always converted to invariant lowercase.

`host` HostString

The host portion of the uri normally included in the Host header. This may include the port.

`pathBase` PathString

The first portion of the request path associated with application root.

`path` PathString

The portion of the request path that identifies the requested resource.

`query` QueryString

The query, if any.

Returns

string

The combined URI components, properly encoded for use in HTTP headers.

BuildAbsolute(CaseHandling, string, HostString, PathString, PathString, QueryString)

Combines the given URI components into a string that is properly encoded for use in HTTP headers. Note that unicode in the HostString will be encoded as punycode.

```
public static string BuildAbsolute(CaseHandlingUriBuilder.CaseHandling handling, string
scheme, HostString host, PathString pathBase = default, PathString path = default,
QueryString query = default)
```

Parameters

`handling` CaseHandlingUriBuilder.CaseHandling

Determines case handling for the result. `query` is always converted to invariant lowercase.

`scheme` string

http, https, etc.

`host` HostString

The host portion of the uri normally included in the Host header. This may include the port.

`pathBase` PathString

The first portion of the request path associated with application root.

`path` PathString

The portion of the request path that identifies the requested resource.

`query` QueryString

The query, if any.

Returns

string

The combined URI components, properly encoded for use in HTTP headers.

BuildRelative(CaseHandling, PathString, PathString, QueryString)

Combines the given URI components into a string that is properly encoded for use in HTTP headers.

```
public static string BuildRelative(CaseHandlingUriBuilder.CaseHandling handling, PathString
pathBase = default, PathString path = default, QueryString query = default)
```

Parameters

`handling` CaseHandlingUriBuilder.CaseHandling

Determines case handling for the result. `query` is always converted to invariant lowercase.

`pathBase` PathString

The first portion of the request path associated with application root.

`path` PathString

The portion of the request path that identifies the requested resource.

`query` QueryString

The query, if any.

Returns

string

The combined URI components, properly encoded for use in HTTP headers.

Encode(CaseHandling, string)

Generates a string from the given absolute or relative Uri that is appropriately encoded for use in HTTP headers. Note that a unicode host name will be encoded as punycode.

```
public static string Encode(CaseHandlingUriBuilder.CaseHandling handling, string uri)
```

Parameters

`handling` CaseHandlingUriBuilder.CaseHandling

Determines case handling for the result.

`uri` string

The Uri to encode.

Returns

string

The encoded string version of `uri`.

Encode(CaseHandling, Uri)

Generates a string from the given absolute or relative Uri that is appropriately encoded for use in HTTP headers. Note that a unicode host name will be encoded as punycode.

```
public static string Encode(CaseHandlingUriBuilder.CaseHandling handling, Uri uri)
```

Parameters

`handling` CaseHandlingUriBuilder.CaseHandling

Determines case handling for the result.

`uri` Uri

The Uri to encode.

Returns

string

The encoded string version of `uri`.

Enum CaseHandlingUriBuilder.CaseHandling

Namespace: SixLabors.ImageSharp.Web

Assembly: SixLabors.ImageSharp.Web.dll

Provides Uri case handling options.

```
public enum CaseHandlingUriBuilder.CaseHandling
```

Fields

`LowerInvariant = 1`

All URI components are converted to lower case using the invariant culture before combining.

`None = 0`

No adjustments to casing are made.

Enum CommandHandling

Namespace: SixLabors.ImageSharp.Web

Assembly: SixLabors.ImageSharp.Web.dll

Provides enumeration for handling CommandCollection instances when processing a request.

```
public enum CommandHandling
```

Fields

`None = 1`

The command collection will be processed unaltered.

`Sanitize = 0`

The command collection will be stripped of any unknown commands.

Class ExifOrientationUtilities

Namespace: SixLabors.ImageSharp.Web

Assembly: SixLabors.ImageSharp.Web.dll

Contains various helper methods for working with EXIF orientation values.

```
public static class ExifOrientationUtilities
```

Inheritance

object ← ExifOrientationUtilities

Inherited Members

object.Equals(object) , object.Equals(object, object) , object.GetHashCode() , object.GetType() , object.MemberwiseClone() , object.ReferenceEquals(object, object) , object.ToString()

Methods

IsExifOrientationRotated(ushort)

Returns a value indicating whether an EXIF orientation is rotated (not flipped).

```
public static bool IsExifOrientationRotated(ushort orientation)
```

Parameters

`orientation` ushort

The EXIF orientation.

Returns

bool

true if the orientation value indicates rotation; otherwise false.

Transform(AnchorPositionMode, ushort)

Transforms the specified anchor value depending on the specified `orientation`.

```
public static AnchorPositionMode Transform(AnchorPositionMode anchor, ushort orientation)
```

Parameters

`anchor` AnchorPositionMode

The input anchor value.

`orientation` ushort

The EXIF orientation.

Returns

AnchorPositionMode

The transformed anchor.

Transform(Size, ushort)

Transforms the specified size value depending on the specified `orientation`.

```
public static Size Transform(Size size, ushort orientation)
```

Parameters

`size` Size

The input size value.

`orientation` ushort

The EXIF orientation.

Returns

Size

The transformed size.

Transform(Vector2, Vector2, Vector2, ushort)

Transforms the specified vector value depending on the specified `orientation`.

```
public static Vector2 Transform(Vector2 position, Vector2 min, Vector2 max,
ushort orientation)
```

Parameters

`position` Vector2

The input vector value.

`min` Vector2

The minimum bounds of the area of interest.

`max` Vector2

The maximum bounds of the area of interest.

`orientation` ushort

The EXIF orientation.

Returns

Vector2

The transformed vector.

Class Format

Namespace: SixLabors.ImageSharp.Web

Assembly: SixLabors.ImageSharp.Web.dll

Contains reusable static instances of known image formats.

```
public static class Format
```

Inheritance

object ← Format

Inherited Members

object.Equals(object) , object.Equals(object, object) , object.GetHashCode() , object.GetType() , object.MemberwiseClone() , object.ReferenceEquals(object, object) , object.ToString()

Properties

Bmp

Gets the Bmp encoding format.

```
public static FormatCommand Bmp { get; }
```

Property Value

FormatCommand

Gif

Gets the Gif encoding format.

```
public static FormatCommand Gif { get; }
```

Property Value

FormatCommand

Jpg

Gets the Jpg encoding format.

```
public static FormatCommand Jpg { get; }
```

Property Value

FormatCommand

Png

Gets the Png encoding format.

```
public static FormatCommand Png { get; }
```

Property Value

FormatCommand

Tga

Gets the Tga encoding format.

```
public static FormatCommand Tga { get; }
```

Property Value

FormatCommand

WebP

Gets the WebP encoding format.

```
public static FormatCommand WebP { get; }
```

Property Value

FormatCommand

Struct FormatCommand

Namespace: SixLabors.ImageSharp.Web

Assembly: SixLabors.ImageSharp.Web.dll

Represents a command for setting the type of format with which to encode processed images.

```
public readonly struct FormatCommand
```

Inherited Members
ValueType.Equals(object) , ValueType.GetHashCode() , ValueType.ToString() , object.Equals(object, object) , object.GetType() , object.ReferenceEquals(object, object)

Constructors

FormatCommand(string)

Initializes a new instance of the FormatCommand struct.

```
public FormatCommand(string name)
```

Parameters

`name` string

The name of the resampler command.

Properties

Name

Gets the name of the resampler command.

```
public string Name { get; }
```

Property Value

string

Class FormatUtilities

Namespace: SixLabors.ImageSharp.Web

Assembly: SixLabors.ImageSharp.Web.dll

Contains various helper methods for working with image formats based on the given configuration.

```
public sealed class FormatUtilities
```

Inheritance

object ← FormatUtilities

Inherited Members

object.Equals(object) , object.Equals(object, object) , object.GetHashCode() , object.GetType() , object.ReferenceEquals(object, object) , object.ToString()

Constructors

FormatUtilities(IOptions<ImageSharpMiddlewareOptions>)

Initializes a new instance of the FormatUtilities class.

```
public FormatUtilities(IOptions<ImageSharpMiddlewareOptions> options)
```

Parameters

`options` IOptions<ImageSharpMiddlewareOptions>

The middleware options.

Methods

GetExtensionFromContentType(string)

Gets the correct extension for the given content type (mime-type).

```
public string GetExtensionFromContentType(string contentType)
```

Parameters

`contentType` string

The content type (mime-type).

Returns

string

The string.

TryGetExtensionFromUri(string, out string?)

Gets the file extension for the given image uri.

```
public bool TryGetExtensionFromUri(string uri, out string? extension)
```

Parameters

`uri` string

The full request uri.

`extension` string

When this method returns, contains the file extension for the image source, if the path exists; otherwise, the default value for the type of the path parameter. This parameter is passed uninitialized.

Returns

bool

true if the uri contains an extension; otherwise, false.

Class FormattedImage

Namespace: SixLabors.ImageSharp.Web

Assembly: SixLabors.ImageSharp.Web.dll

A class encapsulating an image with a particular file encoding.

```
public sealed class FormattedImage : IDisposable
```

Inheritance

object ← FormattedImage

Implements

IDisposable

Inherited Members

object.Equals(object) , object.Equals(object, object) , object.GetHashCode() , object.GetType() , object.ReferenceEquals(object, object) , object.ToString()

Constructors

FormattedImage(Image, IImageFormat)

Initializes a new instance of the FormattedImage class.

```
public FormattedImage(Image image, IImageFormat format)
```

Parameters

`image` Image

The image.

`format` IImageFormat

The format.

Properties

Encoder

Gets or sets the encoder.

```
public IImageEncoder Encoder { get; set; }
```

Property Value

IImageEncoder

Format

Gets or sets the format.

```
public IImageFormat Format { get; set; }
```

Property Value

IImageFormat

Image

Gets the decoded image.

```
public Image Image { get; }
```

Property Value

Image

Methods

Dispose()

Performs application-defined tasks associated with freeing, releasing, or resetting unmanaged resources.

```
public void Dispose()
```

TryGetExifOrientation(out ushort)

Gets the EXIF orientation metadata for the FormattedImage.

```
public bool TryGetExifOrientation(out ushort value)
```

Parameters

`value` ushort

When this method returns, contains the value parsed from decoded EXIF metadata; otherwise, the default value for the type of the `value` parameter. This parameter is passed uninitialized. Use ExifOrientationMode for comparison.

Returns

bool

true if the FormattedImage contains EXIF orientation metadata for Orientation; otherwise, false.

See Also

IDisposable

Class HMACUtilities

Namespace: SixLabors.ImageSharp.Web

Assembly: SixLabors.ImageSharp.Web.dll

Provides methods to compute a Hash-based Message Authentication Code (HMAC).

```
public static class HMACUtilities
```

Inheritance

object ← HMACUtilities

Inherited Members

object.Equals(object) , object.Equals(object, object) , object.GetHashCode() , object.GetType() , object.MemberwiseClone() , object.ReferenceEquals(object, object) , object.ToString()

Fields

TokenCommand

The command used by image requests for transporting Hash-based Message Authentication Code (HMAC) tokens.

```
public const string TokenCommand = "hmac"
```

Field Value

string

Methods

ComputeHMACSHA256(string, byte[])

Computes a Hash-based Message Authentication Code (HMAC) by using the SHA256 hash function.

```
public static string ComputeHMACSHA256(string value, byte[] secret)
```

Parameters

`value` string

The value to hash

`secret` byte[]

The secret key for HMACSHA256 encryption. The key can be any length. However, the recommended size is 64 bytes.

Returns

string

The hashed string.

ComputeHMACSHA384(string, byte[])

Computes a Hash-based Message Authentication Code (HMAC) by using the SHA384 hash function.

```
public static string ComputeHMACSHA384(string value, byte[] secret)
```

Parameters

`value` string

The value to hash

`secret` byte[]

The secret key for HMACSHA256 encryption. The key can be any length. However, the recommended size is 128 bytes.

Returns

string

The hashed string.

ComputeHMACSHA512(string, byte[])

Computes a Hash-based Message Authentication Code (HMAC) by using the SHA512 hash function.

```
public static string ComputeHMACSHA512(string value, byte[] secret)
```

Parameters

`value` string

The value to hash

`secret` byte[]

The secret key for HMACSHA256 encryption. The key can be any length. However, the recommended size is 128 bytes.

Returns

string

The hashed string.

Interface IImageSharpBuilder

Namespace: SixLabors.ImageSharp.Web

Assembly: SixLabors.ImageSharp.Web.dll

Defines a contract for adding ImageSharp services.

```
public interface IImageSharpBuilder
```

Extension Methods

ImageSharpBuilderExtensions.AddConverter<TConverter>(IImageSharpBuilder) ,
ImageSharpBuilderExtensions.AddConverter<TConverter>(IImageSharpBuilder,
Func<IServiceProvider, TConverter>) ,
ImageSharpBuilderExtensions.AddProcessor<T>(IImageSharpBuilder) ,
ImageSharpBuilderExtensions.AddProcessor<TProcessor>(IImageSharpBuilder,
Func<IServiceProvider, TProcessor>) ,
ImageSharpBuilderExtensions.AddProvider<TProvider>(IImageSharpBuilder) ,
ImageSharpBuilderExtensions.AddProvider<TProvider>(IImageSharpBuilder,
Func<IServiceProvider, TProvider>) ,
ImageSharpBuilderExtensions.ClearConverters(IImageSharpBuilder) ,
ImageSharpBuilderExtensions.ClearProcessors(IImageSharpBuilder) ,
ImageSharpBuilderExtensions.ClearProviders(IImageSharpBuilder) ,
ImageSharpBuilderExtensions.Configure<TOptions>(IImageSharpBuilder, IConfiguration) ,
ImageSharpBuilderExtensions.Configure<TOptions>(IImageSharpBuilder, Action<TOptions>) ,
ImageSharpBuilderExtensions.InsertProvider<TProvider>(IImageSharpBuilder, int) ,
ImageSharpBuilderExtensions.InsertProvider<TProvider>(IImageSharpBuilder, int,
Func<IServiceProvider, TProvider>) ,
ImageSharpBuilderExtensions.RemoveConverter<TConverter>(IImageSharpBuilder) ,
ImageSharpBuilderExtensions.RemoveProcessor<TProcessor>(IImageSharpBuilder) ,
ImageSharpBuilderExtensions.RemoveProvider<TProvider>(IImageSharpBuilder) ,
ImageSharpBuilderExtensions.SetCache(IImageSharpBuilder, Func<IServiceProvider,
IImageCache>) ,
ImageSharpBuilderExtensions.SetCacheHash(IImageSharpBuilder, Func<IServiceProvider,
ICacheHash>) ,
ImageSharpBuilderExtensions.SetCacheHash<TCacheHash>(IImageSharpBuilder) ,
ImageSharpBuilderExtensions.SetCacheKey(IImageSharpBuilder, Func<IServiceProvider,
ICacheKey>) ,
ImageSharpBuilderExtensions.SetCacheKey<TCacheKey>(IImageSharpBuilder) ,

ImageSharpBuilderExtensions.SetCache<TCache>(IImageSharpBuilder) ,
ImageSharpBuilderExtensions.SetRequestParser(IImageSharpBuilder, Func<IServiceProvider,
IRequestParser>) ,
ImageSharpBuilderExtensions.SetRequestParser<TParser>(IImageSharpBuilder)

Properties

Services

Gets the IServiceCollection where ImageSharp services are configured.

```
IServiceCollection Services { get; }
```

Property Value

IServiceCollection

Struct ImageCacheMetadata

Namespace: SixLabors.ImageSharp.Web

Assembly: SixLabors.ImageSharp.Web.dll

Represents the metadata associated with an image file.

```
public readonly struct ImageCacheMetadata : IEquatable<ImageCacheMetadata>
```

Implements

IEquatable<ImageCacheMetadata>

Inherited Members

object.Equals(object, object) , object.GetType() , object.ReferenceEquals(object, object)

Constructors

ImageCacheMetadata(DateTime, DateTime, string, TimeSpan, long)

Initializes a new instance of the ImageCacheMetadata struct.

```
public ImageCacheMetadata(DateTime sourceLastWriteTimeUtc, DateTime cacheLastWriteTimeUtc,
string contentType, TimeSpan cacheControlMaxAge, long contentLength)
```

Parameters

`sourceLastWriteTimeUtc` DateTime

The date and time in coordinated universal time (UTC) since the source file was last modified.

`cacheLastWriteTimeUtc` DateTime

The date and time in coordinated universal time (UTC) since the cache file was last modified.

`contentType` string

The content type for the source file.

`cacheControlMaxAge` TimeSpan

The maximum amount of time a resource will be considered fresh.

`contentLength` long

The length of the image in bytes.

Properties

CacheControlMaxAge

Gets the maximum amount of time a resource will be considered fresh.

```
public TimeSpan CacheControlMaxAge { get; }
```

Property Value

TimeSpan

CacheLastWriteTimeUtc

Gets the date and time in coordinated universal time (UTC) since the cached file was last modified.

```
public DateTime CacheLastWriteTimeUtc { get; }
```

Property Value

DateTime

ContentLength

Gets the length of the image in bytes.

```
public long ContentLength { get; }
```

Property Value

long

ContentType

Gets the content type of the source file.

```
public string ContentType { get; }
```

Property Value

string

SourceLastWriteTimeUtc

Gets the date and time in coordinated universal time (UTC) since the source file was last modified.

```
public DateTime SourceLastWriteTimeUtc { get; }
```

Property Value

DateTime

Methods

Equals(ImageCacheMetadata)

Indicates whether the current object is equal to another object of the same type.

```
public bool Equals(ImageCacheMetadata other)
```

Parameters

`other` ImageCacheMetadata

An object to compare with this object.

Returns

bool

true if the current object is equal to the `other` parameter; otherwise, false.

Equals(object?)

Indicates whether this instance and a specified object are equal.

```
public override bool Equals(object? obj)
```

Parameters

`obj` object

The object to compare with the current instance.

Returns

bool

true if `obj` and this instance are the same type and represent the same value; otherwise, false.

FromDictionary(IDictionary<string, string>)

Returns a new ImageCacheMetadata parsed from the given dictionary.

```
public static ImageCacheMetadata FromDictionary(IDictionary<string, string> dictionary)
```

Parameters

`dictionary` IDictionary<string, string>

The dictionary to parse and return the metadata from.

Returns

ImageCacheMetadata

The ImageCacheMetadata.

GetHashCode()

Returns the hash code for this instance.

```
public override int GetHashCode()
```

Returns

int

A 32-bit signed integer that is the hash code for this instance.

ReadAsync(Stream)

Asynchronously reads and returns an ImageCacheMetadata from the input stream.

```
public static Task<ImageCacheMetadata> ReadAsync(Stream stream)
```

Parameters

`stream` Stream

The input stream.

Returns

Task<ImageCacheMetadata>

The ImageCacheMetadata.

ToDictionary()

Returns a new Dictionary<TKey, TValue> representing the current instance.

```
public Dictionary<string, string> ToDictionary()
```

Returns

Dictionary<string, string>

The Dictionary<TKey, TValue>.

ToString()

Returns the fully qualified type name of this instance.

```
public override string ToString()
```

Returns

string

The fully qualified type name.

WriteAsync(Stream)

Asynchronously writes the metadata to the target stream.

```
public Task WriteAsync(Stream stream)
```

Parameters

`stream` Stream

The target stream.

Returns

Task

A Task representing the asynchronous operation.

Operators

operator ==(in ImageCacheMetadata, in ImageCacheMetadata)

Compares two ImageCacheMetadata objects for equality.

```
public static bool operator ==(in ImageCacheMetadata left, in ImageCacheMetadata right)
```

Parameters

`left` ImageCacheMetadata

The ImageCacheMetadata on the left side of the operand.

`right` ImageCacheMetadata

The ImageCacheMetadata on the right side of the operand.

Returns

bool

True if the current left is equal to the `right` parameter; otherwise, false.

operator !=(in ImageCacheMetadata, in ImageCacheMetadata)

Compares two ImageCacheMetadata objects for inequality.

```
public static bool operator !=(in ImageCacheMetadata left, in ImageCacheMetadata right)
```

Parameters

`left` ImageCacheMetadata

The ImageCacheMetadata on the left side of the operand.

`right` ImageCacheMetadata

The ImageCacheMetadata on the right side of the operand.

Returns

bool

True if the current left is unequal to the `right` parameter; otherwise, false.

Struct ImageMetadata

Namespace: SixLabors.ImageSharp.Web

Assembly: SixLabors.ImageSharp.Web.dll

Represents the metadata associated with an image file.

```
public readonly struct ImageMetadata : IEquatable<ImageMetadata>
```

Implements
IEquatable<ImageMetadata>

Inherited Members
object.Equals(object, object) , object.GetType() , object.ReferenceEquals(object, object)

Constructors

ImageMetadata(DateTime, long)

Initializes a new instance of the ImageMetadata struct.

```
public ImageMetadata(DateTime lastWriteTimeUtc, long contentLength)
```

Parameters

`lastWriteTimeUtc` DateTime

The date and time in coordinated universal time (UTC) since the source file was last modified.

`contentLength` long

The length of the image in bytes.

ImageMetadata(DateTime, TimeSpan, long)

Initializes a new instance of the ImageMetadata struct.

```
public ImageMetadata(DateTime lastWriteTimeUtc, TimeSpan cacheControlMaxAge,
long contentLength)
```

Parameters

`lastWriteTimeUtc` DateTime

The date and time in coordinated universal time (UTC) since the source file was last modified.

`cacheControlMaxAge` TimeSpan

The maximum amount of time a resource will be considered fresh.

`contentLength` long

The length of the image in bytes.

Properties

CacheControlMaxAge

Gets the maximum amount of time a resource will be considered fresh.

```
public TimeSpan CacheControlMaxAge { get; }
```

Property Value

TimeSpan

ContentLength

Gets the length of the image in bytes.

```
public long ContentLength { get; }
```

Property Value

long

LastWriteTimeUtc

Gets the date and time in coordinated universal time (UTC) since the source file was last modified.

```
public DateTime LastWriteTimeUtc { get; }
```

Property Value

DateTime

Methods

Equals(ImageMetadata)

Indicates whether the current object is equal to another object of the same type.

```
public bool Equals(ImageMetadata other)
```

Parameters

`other` ImageMetadata

An object to compare with this object.

Returns

bool

true if the current object is equal to the `other` parameter; otherwise, false.

Equals(object?)

Indicates whether this instance and a specified object are equal.

```
public override bool Equals(object? obj)
```

Parameters

obj object

The object to compare with the current instance.

Returns

bool

true if `obj` and this instance are the same type and represent the same value; otherwise, false.

GetHashCode()

Returns the hash code for this instance.

```
public override int GetHashCode()
```

Returns

int

A 32-bit signed integer that is the hash code for this instance.

ToString()

Returns the fully qualified type name of this instance.

```
public override string ToString()
```

Returns

string

The fully qualified type name.

Operators

operator ==(in ImageMetadata, in ImageMetadata)

Compares two ImageMetadata objects for equality.

```
public static bool operator ==(in ImageMetadata left, in ImageMetadata right)
```

Parameters

`left` ImageMetadata

The ImageMetadata on the left side of the operand.

`right` ImageMetadata

The ImageMetadata on the right side of the operand.

Returns

bool

True if the current left is equal to the `right` parameter; otherwise, false.

operator !=(in ImageMetadata, in ImageMetadata)

Compares two ImageMetadata objects for inequality.

```
public static bool operator !=(in ImageMetadata left, in ImageMetadata right)
```

Parameters

`left` ImageMetadata

The ImageMetadata on the left side of the operand.

`right` ImageMetadata

The ImageMetadata on the right side of the operand.

Returns

bool

True if the current left is unequal to the `right` parameter; otherwise, false.

Class ImageSharpBuilderExtensions

Namespace: SixLabors.ImageSharp.Web

Assembly: SixLabors.ImageSharp.Web.dll

Extension methods for IImageSharpBuilder that allow configuration of services.

```
public static class ImageSharpBuilderExtensions
```

Inheritance

object ← ImageSharpBuilderExtensions

Inherited Members

object.Equals(object) , object.Equals(object, object) , object.GetHashCode() , object.GetType() , object.MemberwiseClone() , object.ReferenceEquals(object, object) , object.ToString()

Methods

AddConverter<TConverter>(IImageSharpBuilder)

Adds the given ICommandConverter to the converter collection within the service collection.

```
public static IImageSharpBuilder AddConverter<TConverter>(this IImageSharpBuilder builder)
where TConverter : class, ICommandConverter
```

Parameters

`builder` IImageSharpBuilder

The core builder.

Returns

IImageSharpBuilder

The IImageSharpBuilder.

Type Parameters

`TConverter`

The type of class implementing ICommandConverter to add.

AddConverter<TConverter>(IImageSharpBuilder, Func<IServiceProvider, TConverter>)

Adds the given ICommandConverter to the converter collection within the service collection.

```
public static IImageSharpBuilder AddConverter<TConverter>(this IImageSharpBuilder
builder, Func<IServiceProvider, TConverter> implementationFactory) where TConverter :
class, ICommandConverter
```

Parameters

`builder` IImageSharpBuilder

The core builder.

`implementationFactory` Func<IServiceProvider, TConverter>

The factory method for returning a ICommandConverter.

Returns

IImageSharpBuilder

The IImageSharpBuilder.

Type Parameters

`TConverter`

The type of class implementing ICommandConverter to add.

AddProcessor<T>(IImageSharpBuilder)

Adds the given IImageWebProcessor to the processor collection within the service collection.

```
public static IImageSharpBuilder AddProcessor<T>(this IImageSharpBuilder builder) where T :
class, IImageWebProcessor
```

Parameters

`builder` IImageSharpBuilder

The core builder.

Returns

IImageSharpBuilder

The IImageSharpBuilder.

Type Parameters

`T`

The type of class implementing IImageWebProcessor to add.

AddProcessor<TProcessor>(IImageSharpBuilder, Func<IServiceProvider, TProcessor>)

Adds the given IImageWebProcessor to the processor collection within the service collection.

```
public static IImageSharpBuilder AddProcessor<TProcessor>(this IImageSharpBuilder
builder, Func<IServiceProvider, TProcessor> implementationFactory) where TProcessor :
class, IImageWebProcessor
```

Parameters

`builder` IImageSharpBuilder

The core builder.

`implementationFactory` Func<IServiceProvider, TProcessor>

The factory method for returning a IImageProvider.

Returns

IImageSharpBuilder

The IImageSharpBuilder.

Type Parameters

`TProcessor`

The type of class implementing IImageWebProcessor to add.

AddProvider<TProvider>(IImageSharpBuilder)

Adds the given IImageProvider to the provider collection within the service collection.

```
public static IImageSharpBuilder AddProvider<TProvider>(this IImageSharpBuilder builder)
where TProvider : class, IImageProvider
```

Parameters

`builder` IImageSharpBuilder

The core builder.

Returns

IImageSharpBuilder

The IImageSharpBuilder.

Type Parameters

`TProvider`

The type of class implementing IImageProvider to add.

AddProvider<TProvider>(IImageSharpBuilder, Func<IServiceProvider, TProvider>)

Adds the given IImageProvider to the provider collection within the service collection.

```
public static IImageSharpBuilder AddProvider<TProvider>(this IImageSharpBuilder
builder, Func<IServiceProvider, TProvider> implementationFactory) where TProvider :
class, IImageProvider
```

Parameters

`builder` IImageSharpBuilder

The core builder.

`implementationFactory` Func<IServiceProvider, TProvider>

The factory method for returning a IImageProvider.

Returns

IImageSharpBuilder

The IImageSharpBuilder.

Type Parameters

`TProvider`

The type of class implementing IImageProvider to add.

ClearConverters(IImageSharpBuilder)

Removes all ICommandConverter instances from the converter collection within the service collection.

```
public static IImageSharpBuilder ClearConverters(this IImageSharpBuilder builder)
```

Parameters

`builder` IImageSharpBuilder

The core builder.

Returns

IImageSharpBuilder

The IImageSharpBuilder.

ClearProcessors(IImageSharpBuilder)

Removes all IImageWebProcessor instances from the processor collection within the service collection.

```
public static IImageSharpBuilder ClearProcessors(this IImageSharpBuilder builder)
```

Parameters

`builder` IImageSharpBuilder

The core builder.

Returns

IImageSharpBuilder

The IImageSharpBuilder.

ClearProviders(IImageSharpBuilder)

Removes all IImageProvider instances from the provider collection within the service collection.

```
public static IImageSharpBuilder ClearProviders(this IImageSharpBuilder builder)
```

Parameters

`builder` IImageSharpBuilder

The core builder.

Returns

IImageSharpBuilder

The IImageSharpBuilder.

Configure<TOptions>(IImageSharpBuilder, IConfiguration)

Registers an action used to configure a particular type of options.

```
[RequiresDynamicCode("Binding strongly typed objects to configuration values may require
generating dynamic code at runtime.")]
[RequiresUnreferencedCode("TOptions's dependent types may have their members trimmed. Ensure
all required members are preserved.")]
public static IImageSharpBuilder Configure<TOptions>(this IImageSharpBuilder builder,
IConfiguration config) where TOptions : class
```

Parameters

`builder` IImageSharpBuilder

The core builder.

`config` IConfiguration

The configuration being bound.

Returns

IImageSharpBuilder

The IImageSharpBuilder.

Type Parameters

`TOptions`

The options type to be configured.

Configure<TOptions>(IImageSharpBuilder, Action<TOptions>)

Registers an action used to configure a particular type of options.

```
public static IImageSharpBuilder Configure<TOptions>(this IImageSharpBuilder builder,
Action<TOptions> configureOptions) where TOptions : class
```

Parameters

`builder` IImageSharpBuilder

The core builder.

`configureOptions` Action<TOptions>

The action used to configure the options.

Returns

IImageSharpBuilder

The IImageSharpBuilder.

Type Parameters

`TOptions`

The options type to be configured.

InsertProvider<TProvider>(IImageSharpBuilder, int)

Inserts the given IImageProvider at the give index into to the provider collection within the service collection.

```
public static IImageSharpBuilder InsertProvider<TProvider>(this IImageSharpBuilder builder,
int index) where TProvider : class, IImageProvider
```

Parameters

`builder` IImageSharpBuilder

The core builder.

`index` int

The zero-based index at which the provider should be inserted.

Returns

IImageSharpBuilder

The IImageSharpBuilder.

Type Parameters

`TProvider`

The type of class implementing IImageProvider to add.

InsertProvider<TProvider>(IImageSharpBuilder, int, Func<IServiceProvider, TProvider>)

Inserts the given IImageProvider at the give index into the provider collection within the service collection.

```
public static IImageSharpBuilder InsertProvider<TProvider>(this IImageSharpBuilder builder,
int index, Func<IServiceProvider, TProvider> implementationFactory) where TProvider :
class, IImageProvider
```

Parameters

`builder` IImageSharpBuilder

The core builder.

`index` int

The zero-based index at which the provider should be inserted.

`implementationFactory` Func<IServiceProvider, TProvider>

The factory method for returning a IImageProvider.

Returns

IImageSharpBuilder

The IImageSharpBuilder.

Type Parameters

`TProvider`

The type of class implementing IImageProvider to add.

RemoveConverter<TConverter>(IImageSharpBuilder)

Removes the given ICommandConverter from the converter collection within the service collection.

```
public static IImageSharpBuilder RemoveConverter<TConverter>(this IImageSharpBuilder
builder) where TConverter : class, ICommandConverter
```

Parameters

`builder` IImageSharpBuilder

The core builder.

Returns

IImageSharpBuilder

The IImageSharpBuilder.

Type Parameters

`TConverter`

The type of class implementing ICommandConverter to add.

RemoveProcessor<TProcessor>(IImageSharpBuilder)

Removes the given IImageWebProcessor from the processor collection within the service collection.

```
public static IImageSharpBuilder RemoveProcessor<TProcessor>(this IImageSharpBuilder
builder) where TProcessor : class, IImageWebProcessor
```

Parameters

builder IImageSharpBuilder

The core builder.

Returns

IImageSharpBuilder

The IImageSharpBuilder.

Type Parameters

TProcessor

The type of class implementing IImageWebProcessor to add.

RemoveProvider<TProvider>(IImageSharpBuilder)

Removes the given IImageProvider from the provider collection within the service collection.

```
public static IImageSharpBuilder RemoveProvider<TProvider>(this IImageSharpBuilder builder)
where TProvider : class, IImageProvider
```

Parameters

builder IImageSharpBuilder

The core builder.

Returns

IImageSharpBuilder

The IImageSharpBuilder.

Type Parameters

`TProvider`

The type of class implementing IImageProvider to add.

SetCache(IImageSharpBuilder, Func<IServiceProvider, IImageCache>)

Sets the given IImageCache adding it to the service collection.

```
public static IImageSharpBuilder SetCache(this IImageSharpBuilder builder,
Func<IServiceProvider, IImageCache> implementationFactory)
```

Parameters

`builder` IImageSharpBuilder

The core builder.

`implementationFactory` Func<IServiceProvider, IImageCache>

The factory method for returning a IImageCache.

Returns

IImageSharpBuilder

The IImageSharpBuilder.

SetCacheHash(IImageSharpBuilder, Func<IServiceProvider, ICacheHash>)

Sets the given ICacheHash adding it to the service collection.

```
public static IImageSharpBuilder SetCacheHash(this IImageSharpBuilder builder,
```

```
    Func<IServiceProvider, ICacheHash> implementationFactory)
```

Parameters

`builder` IImageSharpBuilder

The core builder.

`implementationFactory` Func<IServiceProvider, ICacheHash>

The factory method for returning a ICacheHash.

Returns

IImageSharpBuilder

The IImageSharpBuilder.

SetCacheHash<TCacheHash>(IImageSharpBuilder)

Sets the given ICacheHash adding it to the service collection.

```
public static IImageSharpBuilder SetCacheHash<TCacheHash>(this IImageSharpBuilder builder)
where TCacheHash : class, ICacheHash
```

Parameters

`builder` IImageSharpBuilder

The core builder.

Returns

IImageSharpBuilder

The IImageSharpBuilder.

Type Parameters

`TCacheHash`

The type of class implementing ICacheHash to add.

SetCacheKey(IImageSharpBuilder, Func<IServiceProvider, ICacheKey>)

Sets the given ICacheKey adding it to the service collection.

```
public static IImageSharpBuilder SetCacheKey(this IImageSharpBuilder builder,
Func<IServiceProvider, ICacheKey> implementationFactory)
```

Parameters

`builder` IImageSharpBuilder

The core builder.

`implementationFactory` Func<IServiceProvider, ICacheKey>

The factory method for returning a ICacheKey.

Returns

IImageSharpBuilder

The IImageSharpBuilder.

SetCacheKey<TCacheKey>(IImageSharpBuilder)

Sets the given ICacheKey adding it to the service collection.

```
public static IImageSharpBuilder SetCacheKey<TCacheKey>(this IImageSharpBuilder builder)
where TCacheKey : class, ICacheKey
```

Parameters

`builder` IImageSharpBuilder

The core builder.

Returns

IImageSharpBuilder

The IImageSharpBuilder.

Type Parameters

TCacheKey

The type of class implementing ICacheKey to add.

SetCache<TCache>(IImageSharpBuilder)

Sets the given IImageCache adding it to the service collection.

```
public static IImageSharpBuilder SetCache<TCache>(this IImageSharpBuilder builder) where
TCache : class, IImageCache
```

Parameters

builder IImageSharpBuilder

The core builder.

Returns

IImageSharpBuilder

The IImageSharpBuilder.

Type Parameters

TCache

The type of class implementing IImageCache to add.

SetRequestParser(IImageSharpBuilder, Func<IServiceProvider, IRequestParser>)

Sets the given IRequestParser adding it to the service collection.

```
public static IImageSharpBuilder SetRequestParser(this IImageSharpBuilder builder,
Func<IServiceProvider, IRequestParser> implementationFactory)
```

Parameters

`builder` IImageSharpBuilder

The core builder.

`implementationFactory` Func<IServiceProvider, IRequestParser>

The factory method for returning a IRequestParser.

Returns

IImageSharpBuilder

The IImageSharpBuilder.

SetRequestParser<TParser>(IImageSharpBuilder)

Sets the given IRequestParser adding it to the service collection.

```
public static IImageSharpBuilder SetRequestParser<TParser>(this IImageSharpBuilder builder)
where TParser : class, IRequestParser
```

Parameters

`builder` IImageSharpBuilder

The core builder.

Returns

IImageSharpBuilder

The IImageSharpBuilder.

Type Parameters

`TParser`

The type of class implementing IRequestParser to add.

Class RequestAuthorizationUtilities

Namespace: SixLabors.ImageSharp.Web

Assembly: SixLabors.ImageSharp.Web.dll

Contains various helper methods for authorizing image requests.

```
public sealed class RequestAuthorizationUtilities
```

Inheritance

object ← RequestAuthorizationUtilities

Inherited Members

object.Equals(object) , object.Equals(object, object) , object.GetHashCode() , object.GetType() , object.ReferenceEquals(object, object) , object.ToString()

Constructors

RequestAuthorizationUtilities(IOptions<ImageSharpMiddlewareOptions>, IRequestParser, IEnumerable<IImageWebProcessor>, CommandParser, IServiceProvider)

Initializes a new instance of the RequestAuthorizationUtilities class.

```
public RequestAuthorizationUtilities(IOptions<ImageSharpMiddlewareOptions> options,
IRequestParser requestParser, IEnumerable<IImageWebProcessor> processors, CommandParser
commandParser, IServiceProvider serviceProvider)
```

Parameters

`options` IOptions<ImageSharpMiddlewareOptions>

The middleware configuration options.

`requestParser` IRequestParser

An IRequestParser instance used to parse image requests for commands.

`processors` IEnumerable<IImageWebProcessor>

A collection of IImageWebProcessor instances used to process images.

`commandParser` CommandParser

The command parser.

`serviceProvider` IServiceProvider

The service provider.

Fields

TokenCommand

The command used by image requests for transporting Hash-based Message Authentication Code (HMAC) tokens.

```
public const string TokenCommand = "hmac"
```

Field Value

string

Methods

ComputeHMAC(HostString, PathString, QueryString, QueryCollection, CommandHandling)

Compute a Hash-based Message Authentication Code (HMAC) for request authentication.

```
public string? ComputeHMAC(HostString host, PathString path, QueryString queryString,
QueryCollection query, CommandHandling handling)
```

Parameters

`host` HostString

The host header.

`path` PathString

The path or pathbase.

`queryString` QueryString

The querystring.

`query` QueryCollection

The query collection.

`handling` CommandHandling

The command collection handling.

Returns

string

The computed HMAC.

ComputeHMAC(HostString, PathString, QueryString, CommandHandling)

Compute a Hash-based Message Authentication Code (HMAC) for request authentication.

```
public string? ComputeHMAC(HostString host, PathString path, QueryString queryString,
CommandHandling handling)
```

Parameters

`host` HostString

The host header.

`path` PathString

The path or pathbase.

`queryString` QueryString

The querystring.

`handling` CommandHandling

The command collection handling.

Returns

string

The computed HMAC.

ComputeHMAC(HttpContext, CommandHandling)

Compute a Hash-based Message Authentication Code (HMAC) for request authentication.

```
public string? ComputeHMAC(HttpContext context, CommandHandling handling)
```

Parameters

`context` HttpContext

The request HTTP context.

`handling` CommandHandling

The command collection handling.

Returns

string

The computed HMAC.

ComputeHMAC(string, CommandHandling)

Compute a Hash-based Message Authentication Code (HMAC) for request authentication.

```
public string? ComputeHMAC(string uri, CommandHandling handling)
```

Parameters

`uri` string

The uri to compute the code from.

`handling` CommandHandling

The command collection handling.

Returns

string

The computed HMAC.

ComputeHMAC(Uri, CommandHandling)

Compute a Hash-based Message Authentication Code (HMAC) for request authentication.

```
public string? ComputeHMAC(Uri uri, CommandHandling handling)
```

Parameters

`uri` Uri

The uri to compute the code from.

`handling` CommandHandling

The command collection handling.

Returns

string

The computed HMAC.

StripUnknownCommands(CommandCollection)

Strips any unknown commands from the command collection.

```
public void StripUnknownCommands(CommandCollection commands)
```

Parameters

`commands` CommandCollection

The unsanitized command collection.

Class Resampler

Namespace: SixLabors.ImageSharp.Web

Assembly: SixLabors.ImageSharp.Web.dll

Contains reusable static instances of known resampling algorithms.

```
public static class Resampler
```

Inheritance

object ← Resampler

Inherited Members

object.Equals(object) , object.Equals(object, object) , object.GetHashCode() , object.GetType() , object.MemberwiseClone() , object.ReferenceEquals(object, object) , object.ToString()

Properties

Bicubic

Gets the Bicubic sampler that implements the bicubic kernel algorithm W(x)

```
public static ResamplerCommand Bicubic { get; }
```

Property Value

ResamplerCommand

Box

Gets the Box sampler that implements the box algorithm. Similar to nearest neighbor when upscaling. When downscaling the pixels will average, merging pixels together.

```
public static ResamplerCommand Box { get; }
```

Property Value

ResamplerCommand

CatmullRom

Gets the Catmull-Rom sampler, a well known standard Cubic Filter often used as a interpolation function

```
public static ResamplerCommand CatmullRom { get; }
```

Property Value

ResamplerCommand

Hermite

Gets the Hermite sampler. A type of smoothed triangular interpolation filter that rounds off strong edges while preserving flat 'color levels' in the original image.

```
public static ResamplerCommand Hermite { get; }
```

Property Value

ResamplerCommand

Lanczos2

Gets the Lanczos kernel sampler that implements smooth interpolation with a radius of 2 pixels. This algorithm provides sharpened results when compared to others when downsampling.

```
public static ResamplerCommand Lanczos2 { get; }
```

Property Value

ResamplerCommand

Lanczos3

Gets the Lanczos kernel sampler that implements smooth interpolation with a radius of 3 pixels
This algorithm provides sharpened results when compared to others when downsampling.

```
public static ResamplerCommand Lanczos3 { get; }
```

Property Value

ResamplerCommand

Lanczos5

Gets the Lanczos kernel sampler that implements smooth interpolation with a radius of 5 pixels
This algorithm provides sharpened results when compared to others when downsampling.

```
public static ResamplerCommand Lanczos5 { get; }
```

Property Value

ResamplerCommand

Lanczos8

Gets the Lanczos kernel sampler that implements smooth interpolation with a radius of 8 pixels
This algorithm provides sharpened results when compared to others when downsampling.

```
public static ResamplerCommand Lanczos8 { get; }
```

Property Value

ResamplerCommand

MitchellNetravali

Gets the Mitchell-Netravali sampler. This seperable cubic algorithm yields a very good equilibrium between detail preservation (sharpness) and smoothness.

```
public static ResamplerCommand MitchellNetravali { get; }
```

Property Value

ResamplerCommand

NearestNeighbor

Gets the Nearest-Neighbour sampler that implements the nearest neighbor algorithm. This uses a very fast, unscaled filter which will select the closest pixel to the new pixels position.

```
public static ResamplerCommand NearestNeighbor { get; }
```

Property Value

ResamplerCommand

Robidoux

Gets the Robidoux sampler. This algorithm developed by Nicolas Robidoux providing a very good equilibrium between detail preservation (sharpness) and smoothness comparable to Mitchell Netravali.

```
public static ResamplerCommand Robidoux { get; }
```

Property Value

ResamplerCommand

RobidouxSharp

Gets the Robidoux Sharp sampler. A sharpened form of the Robidoux sampler

```
public static ResamplerCommand RobidouxSharp { get; }
```

Property Value

ResamplerCommand

Spline

Gets the Spline sampler. A separable cubic algorithm similar to MitchellNetravali but yielding smoother results.

```
public static ResamplerCommand Spline { get; }
```

Property Value

ResamplerCommand

Triangle

Gets the Triangle sampler, otherwise known as Bilinear. This interpolation algorithm can be used where perfect image transformation with pixel matching is impossible, so that one can calculate and assign appropriate intensity values to pixels

```
public static ResamplerCommand Triangle { get; }
```

Property Value

ResamplerCommand

Welch

Gets the Welch sampler. A high speed algorithm that delivers very sharpened results.

```
public static ResamplerCommand Welch { get; }
```

Property Value

ResamplerCommand

Struct ResamplerCommand

Namespace: SixLabors.ImageSharp.Web

Assembly: SixLabors.ImageSharp.Web.dll

Represents a command for setting the type of sampler to use for resampling operations.

```
public readonly struct ResamplerCommand
```

Inherited Members
ValueType.Equals(object) , ValueType.GetHashCode() , ValueType.ToString() , object.Equals(object, object) , object.GetType() , object.ReferenceEquals(object, object)

Constructors

ResamplerCommand(string)

Initializes a new instance of the ResamplerCommand struct.

```
public ResamplerCommand(string name)
```

Parameters

`name` string

The name of the resampler command.

Properties

Name

Gets the name of the resampler command.

```
public string Name { get; }
```

Property Value

string

Class ServiceCollectionExtensions

Namespace: SixLabors.ImageSharp.Web

Assembly: SixLabors.ImageSharp.Web.dll

Extension methods for IServiceCollection to simplify middleware service registration.

```
public static class ServiceCollectionExtensions
```

Inheritance

object ← ServiceCollectionExtensions

Inherited Members

object.Equals(object) , object.Equals(object, object) , object.GetHashCode() , object.GetType() , object.MemberwiseClone() , object.ReferenceEquals(object, object) , object.ToString()

Methods

AddImageSharp(IServiceCollection)

Adds ImageSharp services to the specified IServiceCollection with the default options.

```
public static IImageSharpBuilder AddImageSharp(this IServiceCollection services)
```

Parameters

`services` IServiceCollection

The IServiceCollection to add services to.

Returns

IImageSharpBuilder

An IImageSharpBuilder that can be used to further configure the ImageSharp services.

AddImageSharp(IServiceCollection, Action<ImageSharpMiddlewareOptions>)

Adds ImageSharp services to the specified IServiceCollection with the given options.

```
public static IImageSharpBuilder AddImageSharp(this IServiceCollection services,
Action<ImageSharpMiddlewareOptions> setupAction)
```

Parameters

`services` IServiceCollection

The IServiceCollection to add services to.

`setupAction` Action<ImageSharpMiddlewareOptions>

An Action<T> to configure the provided ImageSharpMiddlewareOptions.

Returns

IImageSharpBuilder

An IImageSharpBuilder that can be used to further configure the ImageSharp services.

Namespace SixLabors.ImageSharp.Web.Caching

Classes

LegacyV1CacheKey
: Maintained for compatibility purposes only this cache key implementation generates the same out as the V1 middleware. If possible, it is recommended to use the UriRelativeLowerInvariantCacheKey.

PhysicalFileSystemCache
: Implements a physical file system based cache.

PhysicalFileSystemCacheOptions
: Configuration options for the PhysicalFileSystemCache.

SHA256CacheHash
: Creates hashed keys for the given inputs hashing them to string of length ranging from 2 to 64. Hashed keys are the result of the SHA256 computation of the input value for the given length. This ensures low collision rates with a shorter file name.

UriAbsoluteCacheKey
: Creates a cache key based on the request host, path and commands.

UriAbsoluteLowerInvariantCacheKey
: Creates a case insensitive cache key based on the request host, path and commands.

UriRelativeCacheKey
: Creates a cache key based on the request path and commands.

UriRelativeLowerInvariantCacheKey
: Creates a case insensitive cache key based on the request path and commands.

Interfaces

ICacheHash
: Defines a contract that allows the creation of hashed file names for storing cached images.

ICacheKey
: Defines a contract that allows the creation of cache keys (used by ICacheHash to create hashed file names for storing cached images).

IImageCache

Specifies the contract for caching images.

Interface ICacheHash

Namespace: SixLabors.ImageSharp.Web.Caching

Assembly: SixLabors.ImageSharp.Web.dll

Defines a contract that allows the creation of hashed file names for storing cached images.

```
public interface ICacheHash
```

Methods

Create(string, uint)

Returns the hashed file name for the cached image file.

```
string Create(string value, uint length)
```

Parameters

`value` string

The input value to hash.

`length` uint

The length of the returned hash without any extensions.

Returns

string

The string.

Interface ICacheKey

Namespace: SixLabors.ImageSharp.Web.Caching

Assembly: SixLabors.ImageSharp.Web.dll

Defines a contract that allows the creation of cache keys (used by ICacheHash to create hashed file names for storing cached images).

```
public interface ICacheKey
```

Methods

Create(HttpContext, CommandCollection)

Creates the cache key based on the specified context and commands.

```
string Create(HttpContext context, CommandCollection commands)
```

Parameters

`context` HttpContext

The HTTP context.

`commands` CommandCollection

The commands.

Returns

string

The cache key.

Interface IImageCache

Namespace: SixLabors.ImageSharp.Web.Caching

Assembly: SixLabors.ImageSharp.Web.dll

Specifies the contract for caching images.

```
public interface IImageCache
```

Methods

GetAsync(string)

Gets the image resolver associated with the specified key.

```
Task<IImageCacheResolver?> GetAsync(string key)
```

Parameters

key string

The cache key.

Returns

Task<IImageCacheResolver>

The IImageResolver.

SetAsync(string, Stream, ImageCacheMetadata)

Sets the value associated with the specified key.

```
Task SetAsync(string key, Stream stream, ImageCacheMetadata metadata)
```

Parameters

`key` string

The cache key.

`stream` Stream

The stream containing the image to store.

`metadata` ImageCacheMetadata

The ImageCacheMetadata associated with the image to store.

Returns

Task

The task.

Class LegacyV1CacheKey

Namespace: SixLabors.ImageSharp.Web.Caching

Assembly: SixLabors.ImageSharp.Web.dll

Maintained for compatibility purposes only this cache key implementation generates the same out as the V1 middleware. If possible, it is recommended to use the UriRelativeLowerInvariantCacheKey.

```
public class LegacyV1CacheKey : ICacheKey
```

Inheritance

object ← LegacyV1CacheKey

Implements

ICacheKey

Inherited Members

object.Equals(object) , object.Equals(object, object) , object.GetHashCode() , object.GetType() , object.MemberwiseClone() , object.ReferenceEquals(object, object) , object.ToString()

Methods

Create(HttpContext, CommandCollection)

Creates the cache key based on the specified context and commands.

```
public string Create(HttpContext context, CommandCollection commands)
```

Parameters

`context` HttpContext

The HTTP context.

`commands` CommandCollection

The commands.

Returns

string

The cache key.

Class PhysicalFileSystemCache

Namespace: SixLabors.ImageSharp.Web.Caching

Assembly: SixLabors.ImageSharp.Web.dll

Implements a physical file system based cache.

```
public class PhysicalFileSystemCache : IImageCache
```

Inheritance

object ← PhysicalFileSystemCache

Implements

IImageCache

Inherited Members

object.Equals(object) , object.Equals(object, object) , object.GetHashCode() , object.GetType() , object.MemberwiseClone() , object.ReferenceEquals(object, object) , object.ToString()

Constructors

PhysicalFileSystemCache(IOptions<PhysicalFileSystemCacheOptions>, IWebHostEnvironment, FormatUtilities)

Initializes a new instance of the PhysicalFileSystemCache class.

```
public PhysicalFileSystemCache(IOptions<PhysicalFileSystemCacheOptions> options,
IWebHostEnvironment environment, FormatUtilities formatUtilities)
```

Parameters

`options` IOptions<PhysicalFileSystemCacheOptions>

The cache configuration options.

`environment` IWebHostEnvironment

The hosting environment the application is running in.

`formatUtilities` FormatUtilities

Contains various format helper methods based on the current configuration.

Methods

GetAsync(string)

Gets the image resolver associated with the specified key.

```
public Task<IImageCacheResolver?> GetAsync(string key)
```

Parameters

`key` string

The cache key.

Returns

Task<IImageCacheResolver>

The IImageResolver.

SetAsync(string, Stream, ImageCacheMetadata)

Sets the value associated with the specified key.

```
public Task SetAsync(string key, Stream stream, ImageCacheMetadata metadata)
```

Parameters

`key` string

The cache key.

`stream` Stream

The stream containing the image to store.

`metadata` ImageCacheMetadata

The ImageCacheMetadata associated with the image to store.

Returns

Task

The task.

Class PhysicalFileSystemCacheOptions

Namespace: SixLabors.ImageSharp.Web.Caching

Assembly: SixLabors.ImageSharp.Web.dll

Configuration options for the PhysicalFileSystemCache.

```
public class PhysicalFileSystemCacheOptions
```

Inheritance

object ← PhysicalFileSystemCacheOptions

Inherited Members

object.Equals(object) , object.Equals(object, object) , object.GetHashCode() , object.GetType() , object.MemberwiseClone() , object.ReferenceEquals(object, object) , object.ToString()

Properties

CacheFolder

Gets or sets the cache folder name.

```
public string CacheFolder { get; set; }
```

Property Value

string

CacheFolderDepth

Gets or sets the depth of the nested cache folders structure to store the images. Defaults to 8.

```
public uint CacheFolderDepth { get; set; }
```

Property Value

uint

CacheRootPath

Gets or sets the optional cache root folder path.

This value can be null, a fully qualified absolute path, or a path relative to the directory that contains the application content files.

If not set, this will default to the directory that contains the web-servable application content files; commonly 'wwwroot'.

```
public string? CacheRootPath { get; set; }
```

Property Value

string

Class SHA256CacheHash

Namespace: SixLabors.ImageSharp.Web.Caching

Assembly: SixLabors.ImageSharp.Web.dll

Creates hashed keys for the given inputs hashing them to string of length ranging from 2 to 64. Hashed keys are the result of the SHA256 computation of the input value for the given length. This ensures low collision rates with a shorter file name.

```
public sealed class SHA256CacheHash : ICacheHash
```

Inheritance

object ← SHA256CacheHash

Implements

ICacheHash

Inherited Members

object.Equals(object) , object.Equals(object, object) , object.GetHashCode() , object.GetType() , object.ReferenceEquals(object, object) , object.ToString()

Constructors

SHA256CacheHash(IOptions<ImageSharpMiddlewareOptions>)

Initializes a new instance of the SHA256CacheHash class.

```
public SHA256CacheHash(IOptions<ImageSharpMiddlewareOptions> options)
```

Parameters

`options` IOptions<ImageSharpMiddlewareOptions>

The middleware configuration options.

Methods

Create(string, uint)

Returns the hashed file name for the cached image file.

```
public string Create(string value, uint length)
```

Parameters

`value` string

The input value to hash.

`length` uint

The length of the returned hash without any extensions.

Returns

string

The string.

Class UriAbsoluteCacheKey

Namespace: SixLabors.ImageSharp.Web.Caching

Assembly: SixLabors.ImageSharp.Web.dll

Creates a cache key based on the request host, path and commands.

```
public class UriAbsoluteCacheKey : ICacheKey
```

Inheritance

object ← UriAbsoluteCacheKey

Implements

ICacheKey

Inherited Members

object.Equals(object) , object.Equals(object, object) , object.GetHashCode() , object.GetType() , object.MemberwiseClone() , object.ReferenceEquals(object, object) , object.ToString()

Methods

Create(HttpContext, CommandCollection)

Creates the cache key based on the specified context and commands.

```
public string Create(HttpContext context, CommandCollection commands)
```

Parameters

`context` HttpContext

The HTTP context.

`commands` CommandCollection

The commands.

Returns

string

The cache key.

Class UriAbsoluteLowerInvariantCacheKey

Namespace: SixLabors.ImageSharp.Web.Caching

Assembly: SixLabors.ImageSharp.Web.dll

Creates a case insensitive cache key based on the request host, path and commands.

```
public class UriAbsoluteLowerInvariantCacheKey : ICacheKey
```

Inheritance

object ← UriAbsoluteLowerInvariantCacheKey

Implements

ICacheKey

Inherited Members

object.Equals(object) , object.Equals(object, object) , object.GetHashCode() , object.GetType() , object.MemberwiseClone() , object.ReferenceEquals(object, object) , object.ToString()

Methods

Create(HttpContext, CommandCollection)

Creates the cache key based on the specified context and commands.

```
public string Create(HttpContext context, CommandCollection commands)
```

Parameters

`context` HttpContext

The HTTP context.

`commands` CommandCollection

The commands.

Returns

string

The cache key.

Class UriRelativeCacheKey

Namespace: SixLabors.ImageSharp.Web.Caching

Assembly: SixLabors.ImageSharp.Web.dll

Creates a cache key based on the request path and commands.

```
public class UriRelativeCacheKey : ICacheKey
```

Inheritance

object ← UriRelativeCacheKey

Implements

ICacheKey

Inherited Members

object.Equals(object) , object.Equals(object, object) , object.GetHashCode() , object.GetType() , object.MemberwiseClone() , object.ReferenceEquals(object, object) , object.ToString()

Methods

Create(HttpContext, CommandCollection)

Creates the cache key based on the specified context and commands.

```
public string Create(HttpContext context, CommandCollection commands)
```

Parameters

`context` HttpContext

The HTTP context.

`commands` CommandCollection

The commands.

Returns

string

The cache key.

Class UriRelativeLowerInvariantCacheKey

Namespace: SixLabors.ImageSharp.Web.Caching

Assembly: SixLabors.ImageSharp.Web.dll

Creates a case insensitive cache key based on the request path and commands.

```
public class UriRelativeLowerInvariantCacheKey : ICacheKey
```

Inheritance

object ← UriRelativeLowerInvariantCacheKey

Implements

ICacheKey

Inherited Members

object.Equals(object) , object.Equals(object, object) , object.GetHashCode() , object.GetType() , object.MemberwiseClone() , object.ReferenceEquals(object, object) , object.ToString()

Methods

Create(HttpContext, CommandCollection)

Creates the cache key based on the specified context and commands.

```
public string Create(HttpContext context, CommandCollection commands)
```

Parameters

`context` HttpContext

The HTTP context.

`commands` CommandCollection

The commands.

Returns

string

The cache key.

Namespace SixLabors.ImageSharp.Web.Commands

Classes

CommandCollection
: Represents an ordered collection of processing commands.

CommandCollectionExtensions
: Extension methods for CommandCollectionExtensions.

CommandParser
: Parses URI derived command values into usable commands for processors.

PresetOnlyQueryCollectionRequestParser
: Parses preset name from the request querystring and returns the commands configured for that preset.

PresetOnlyQueryCollectionRequestParserOptions
: Configuration options for the PresetOnlyQueryCollectionRequestParser.

QueryCollectionRequestParser
: Parses commands from the request querystring.

Interfaces

IRequestParser
: Defines a contract for parsing commands from image requests.

Class CommandCollection

Namespace: SixLabors.ImageSharp.Web.Commands

Assembly: SixLabors.ImageSharp.Web.dll

Represents an ordered collection of processing commands.

```
public sealed class CommandCollection : KeyedCollection<string, KeyValuePair<string, string?
>>, IList<KeyValuePair<string, string?>>, ICollection<KeyValuePair<string, string?>>,
IReadOnlyList<KeyValuePair<string, string?>>, IReadOnlyCollection<KeyValuePair<string,
string?>>, IEnumerable<KeyValuePair<string, string?>>, IList, ICollection, IEnumerable
```

Inheritance

object ← Collection<KeyValuePair<string, string>> ← KeyedCollection<string, KeyValuePair<string, string>> ← CommandCollection

Implements

IList<KeyValuePair<string, string>>, ICollection<KeyValuePair<string, string>>, IReadOnlyList<KeyValuePair<string, string>>, IReadOnlyCollection<KeyValuePair<string, string>>, IEnumerable<KeyValuePair<string, string>>, IList, ICollection, IEnumerable

Inherited Members

KeyedCollection<string, KeyValuePair<string, string>>.Contains(string) ,
KeyedCollection<string, KeyValuePair<string, string>>.Remove(string) ,
KeyedCollection<string, KeyValuePair<string, string>>.TryGetValue(string, out KeyValuePair<string, string>) ,
KeyedCollection<string, KeyValuePair<string, string>>.Comparer ,
KeyedCollection<string, KeyValuePair<string, string>>.this[string] ,
Collection<KeyValuePair<string, string>>.Add(KeyValuePair<string, string>) ,
Collection<KeyValuePair<string, string>>.Clear() ,
Collection<KeyValuePair<string, string>>.Contains(KeyValuePair<string, string>) ,
Collection<KeyValuePair<string, string>>.CopyTo(KeyValuePair<string, string>[], int) ,
Collection<KeyValuePair<string, string>>.GetEnumerator() ,
Collection<KeyValuePair<string, string>>.IndexOf(KeyValuePair<string, string>) ,
Collection<KeyValuePair<string, string>>.Insert(int, KeyValuePair<string, string>) ,
Collection<KeyValuePair<string, string>>.Remove(KeyValuePair<string, string>) ,
Collection<KeyValuePair<string, string>>.RemoveAt(int) ,
Collection<KeyValuePair<string, string>>.Count ,

Collection<KeyValuePair<string, string>>.this[int] , object.Equals(object) , object.Equals(object, object) , object.GetHashCode() , object.GetType() , object.ReferenceEquals(object, object) , object.ToString()

Extension Methods
CommandCollectionExtensions.GetValueOrDefault(CommandCollection, string)

Constructors

CommandCollection()

Initializes a new instance of the CommandCollection class.

```
public CommandCollection()
```

Properties

this[string]

Gets or sets the value associated with the specified key.

```
public string this[string key] { get; set; }
```

Parameters

`key` string

The key of the value to get or set.

Property Value

string

The value associated with the specified key. If the specified key is not found, a get operation throws a KeyNotFoundException, and a set operation creates a new element with the specified key.

Exceptions

ArgumentNullException

`key` is null.

KeyNotFoundException

An element with the specified key does not exist in the collection.

Keys

Gets an IEnumerable<T> representing the keys of the collection.

```
public IEnumerable<string> Keys { get; }
```

Property Value

IEnumerable<string>

Methods

Add(string, string)

Adds an element with the provided key and value to the CommandCollection.

```
public void Add(string key, string value)
```

Parameters

`key` string

The string to use as the key of the element to add.

`value` string

The string to use as the value of the element to add.

Exceptions

ArgumentNullException

`key` is null.

FindIndex(Predicate<string>)

Searches for an element that matches the conditions defined by the specified predicate, and returns the zero-based index of the first occurrence within the entire CommandCollection.

```
public int FindIndex(Predicate<string> match)
```

Parameters

`match` Predicate<string>

The Predicate<T> delegate that defines the conditions of the element to search for.

Returns

int

The zero-based index of the first occurrence of an element that matches the conditions defined by match, if found; otherwise, -1.

Exceptions

ArgumentNullException

`match` is null.

GetKeyForItem(KeyValuePair<string, string?>)

When implemented in a derived class, extracts the key from the specified element.

```
protected override string GetKeyForItem(KeyValuePair<string, string?> item)
```

Parameters

`item` KeyValuePair<string, string>

The element from which to extract the key.

Returns

string

The key for the specified element.

IndexOf(string)

Searches for the specified key and returns the zero-based index of the first occurrence within the entire CommandCollection.

```
public int IndexOf(string key)
```

Parameters

`key` string

The key to locate in the CommandCollection.

Returns

int

The zero-based index of the first occurrence of key within the entire CommandCollection, if found; otherwise, -1.

Insert(int, string, string)

Inserts an element into the CommandCollection at the specified index.

```
public void Insert(int index, string key, string value)
```

Parameters

`index` int

The zero-based index at which item should be inserted.

`key` string

The string to use as the key of the element to insert.

`value` string

The string to use as the value of the element to insert.

Exceptions

ArgumentOutOfRangeException

index is less than zero. -or- index is greater than CommandCollection.Count.

TryGetValue(string, out string?)

Gets the value associated with the specified key.

```
public bool TryGetValue(string key, out string? value)
```

Parameters

`key` string

The key whose value to get.

`value` string

When this method returns, the value associated with the specified key, if the key is found; otherwise, the default value for the type of the value parameter. This parameter is passed uninitialized.

Returns

bool

true if the object that implements CommandCollection contains an element with the specified key; otherwise, false.

Exceptions

ArgumentNullException

`key` is null.

Class CommandCollectionExtensions

Namespace: SixLabors.ImageSharp.Web.Commands

Assembly: SixLabors.ImageSharp.Web.dll

Extension methods for CommandCollectionExtensions.

```
public static class CommandCollectionExtensions
```

Inheritance

object ← CommandCollectionExtensions

Inherited Members

object.Equals(object) , object.Equals(object, object) , object.GetHashCode() , object.GetType() , object.MemberwiseClone() , object.ReferenceEquals(object, object) , object.ToString()

Methods

GetValueOrDefault(CommandCollection, string)

Gets the value associated with the specified key or the default value.

```
public static string? GetValueOrDefault(this CommandCollection collection, string key)
```

Parameters

`collection` CommandCollection

The collection instance.

`key` string

The key of the value to get.

Returns

string

The value associated with the specified key or the default value.

Class CommandParser

Namespace: SixLabors.ImageSharp.Web.Commands

Assembly: SixLabors.ImageSharp.Web.dll

Parses URI derived command values into usable commands for processors.

```
public sealed class CommandParser
```

Inheritance

object ← CommandParser

Inherited Members

object.Equals(object) , object.Equals(object, object) , object.GetHashCode() , object.GetType() , object.ReferenceEquals(object, object) , object.ToString()

Constructors

CommandParser(IEnumerable<ICommandConverter>)

Initializes a new instance of the CommandParser class.

```
public CommandParser(IEnumerable<ICommandConverter> converters)
```

Parameters

`converters` IEnumerable<ICommandConverter>

The collection of command converters.

Methods

ParseValue<T>(string?, CultureInfo)

Parses the given string value converting it to the given type.

```
public T? ParseValue<T>(string? value, CultureInfo culture)
```

Parameters

`value` string

The string value to parse.

`culture` CultureInfo

The CultureInfo to use as the current culture.

Returns

T

The converted instance or the default.

Type Parameters

`T`

The Type to convert the string to.

Interface IRequestParser

Namespace: SixLabors.ImageSharp.Web.Commands

Assembly: SixLabors.ImageSharp.Web.dll

Defines a contract for parsing commands from image requests.

```
public interface IRequestParser
```

Methods

ParseRequestCommands(HttpContext)

Returns a collection of commands from the current request.

```
CommandCollection ParseRequestCommands(HttpContext context)
```

Parameters

`context` HttpContext

Encapsulates all HTTP-specific information about an individual HTTP request.

Returns

CommandCollection

The IDictionary<TKey, TValue>.

Class PresetOnlyQueryCollectionRequestParser

Namespace: SixLabors.ImageSharp.Web.Commands

Assembly: SixLabors.ImageSharp.Web.dll

Parses preset name from the request querystring and returns the commands configured for that preset.

```
public class PresetOnlyQueryCollectionRequestParser : IRequestParser
```

Inheritance

object ← PresetOnlyQueryCollectionRequestParser

Implements

IRequestParser

Inherited Members

object.Equals(object) , object.Equals(object, object) , object.GetHashCode() , object.GetType() , object.MemberwiseClone() , object.ReferenceEquals(object, object) , object.ToString()

Constructors

PresetOnlyQueryCollectionRequestParser(IOptions<PresetOnlyQueryCollectionRequestParserOptions>)

Initializes a new instance of the PresetOnlyQueryCollectionRequestParser class.

```
public
PresetOnlyQueryCollectionRequestParser(IOptions<PresetOnlyQueryCollectionRequestParserOption
s> presetOptions)
```

Parameters

`presetOptions` IOptions<PresetOnlyQueryCollectionRequestParserOptions>

The preset options.

Fields

QueryKey

The command constant for the preset query parameter.

```
public const string QueryKey = "preset"
```

Field Value

string

Methods

ParseRequestCommands(HttpContext)

Returns a collection of commands from the current request.

```
public CommandCollection ParseRequestCommands(HttpContext context)
```

Parameters

`context` HttpContext

Encapsulates all HTTP-specific information about an individual HTTP request.

Returns

CommandCollection

The IDictionary<TKey, TValue>.

Class PresetOnlyQueryCollectionRequestParserOptions

Namespace: SixLabors.ImageSharp.Web.Commands

Assembly: SixLabors.ImageSharp.Web.dll

Configuration options for the PresetOnlyQueryCollectionRequestParser.

```
public class PresetOnlyQueryCollectionRequestParserOptions
```

Inheritance

object ← PresetOnlyQueryCollectionRequestParserOptions

Inherited Members

object.Equals(object) , object.Equals(object, object) , object.GetHashCode() , object.GetType() , object.MemberwiseClone() , object.ReferenceEquals(object, object) , object.ToString()

Properties

Presets

Gets or sets the presets, which is a Dictionary of preset names to command query strings.

```
public Dictionary<string, string> Presets { get; set; }
```

Property Value

Dictionary<string, string>

Class QueryCollectionRequestParser

Namespace: SixLabors.ImageSharp.Web.Commands

Assembly: SixLabors.ImageSharp.Web.dll

Parses commands from the request querystring.

```
public sealed class QueryCollectionRequestParser : IRequestParser
```

Inheritance

object ← QueryCollectionRequestParser

Implements

IRequestParser

Inherited Members

object.Equals(object) , object.Equals(object, object) , object.GetHashCode() , object.GetType() , object.ReferenceEquals(object, object) , object.ToString()

Methods

ParseRequestCommands(HttpContext)

Returns a collection of commands from the current request.

```
public CommandCollection ParseRequestCommands(HttpContext context)
```

Parameters

`context` HttpContext

Encapsulates all HTTP-specific information about an individual HTTP request.

Returns

CommandCollection

The IDictionary<TKey, TValue>.

Namespace SixLabors.ImageSharp.Web.Commands.Converters

Classes

ArrayConverter<T>
Converts the value of a string to a generic array.

ColorConverter
Allows the conversion of strings into rgba32 pixel colors.

EnumConverter
The enum converter. Allows conversion to enumerations.

IntegralNumberConverter<T>
The generic converter for integral types.

ListConverter<T>
Converts the value of a string to a generic list.

SimpleCommandConverter<T>
The generic converter for simple types that implement IConvertible.

Interfaces

ICommandConverter
Defines a contract for converting the value of a string into a different data type.
Implementations should be stateless by design and also implement ICommandConverter<T>.

ICommandConverter<T>
Defines a contract for converting the value of a string into a different data type.
Implementations should be stateless by design and also implement ICommandConverter<T>.

Class ArrayConverter<T>

Namespace: SixLabors.ImageSharp.Web.Commands.Converters

Assembly: SixLabors.ImageSharp.Web.dll

Converts the value of a string to a generic array.

```
public sealed class ArrayConverter<T> : ICommandConverter<T[]>, ICommandConverter
```

Type Parameters

T

The parameter type to convert to.

Inheritance

object ← ArrayConverter<T>

Implements

ICommandConverter<T[]>, ICommandConverter

Inherited Members

object.Equals(object) , object.Equals(object, object) , object.GetHashCode() , object.GetType() , object.ReferenceEquals(object, object) , object.ToString()

Properties

Type

Gets the type this converter returns.

```
public Type Type { get; }
```

Property Value

Type

Methods

ConvertFrom(CommandParser, CultureInfo, string?, Type)

Converts the given string to the type of this converter, using the specified culture information.

```
public T[] ConvertFrom(CommandParser parser, CultureInfo culture, string? value,
Type propertyType)
```

Parameters

`parser` CommandParser

The command parser use for parting commands.

`culture` CultureInfo

The CultureInfo to use as the current parsing culture.

`value` string

The string to convert.

`propertyType` Type

The property type that the converter will convert to.

Returns

T[]

A string that represents the converted value.

Exceptions

NotSupportedException

The conversion cannot be performed.

Class ColorConverter

Namespace: SixLabors.ImageSharp.Web.Commands.Converters

Assembly: SixLabors.ImageSharp.Web.dll

Allows the conversion of strings into rgba32 pixel colors.

```
public sealed class ColorConverter : ICommandConverter<Color>, ICommandConverter
```

Inheritance

object ← ColorConverter

Implements

ICommandConverter<Color>, ICommandConverter

Inherited Members

object.Equals(object) , object.Equals(object, object) , object.GetHashCode() , object.GetType() , object.ReferenceEquals(object, object) , object.ToString()

Properties

Type

Gets the type this converter returns.

```
public Type Type { get; }
```

Property Value

Type

Methods

ConvertFrom(CommandParser, CultureInfo, string?, Type)

Converts the given string to the type of this converter, using the specified culture information.

```
public Color ConvertFrom(CommandParser parser, CultureInfo culture, string? value,
Type propertyType)
```

Parameters

`parser` CommandParser

The command parser use for parting commands.

`culture` CultureInfo

The CultureInfo to use as the current parsing culture.

`value` string

The string to convert.

`propertyType` Type

The property type that the converter will convert to.

Returns

Color

A string that represents the converted value.

Exceptions

NotSupportedException

The conversion cannot be performed.

Class EnumConverter

Namespace: SixLabors.ImageSharp.Web.Commands.Converters

Assembly: SixLabors.ImageSharp.Web.dll

The enum converter. Allows conversion to enumerations.

```
public sealed class EnumConverter : ICommandConverter<object>, ICommandConverter
```

Inheritance

object ← EnumConverter

Implements

ICommandConverter<object>, ICommandConverter

Inherited Members

object.Equals(object) , object.Equals(object, object) , object.GetHashCode() , object.GetType() , object.ReferenceEquals(object, object) , object.ToString()

Properties

Type

Gets the type this converter returns.

```
public Type Type { get; }
```

Property Value

Type

Methods

ConvertFrom(CommandParser, CultureInfo, string?, Type)

Converts the given string to the type of this converter, using the specified culture information.

```
public object? ConvertFrom(CommandParser parser, CultureInfo culture, string? value,
Type propertyType)
```

Parameters

`parser` CommandParser

The command parser use for parting commands.

`culture` CultureInfo

The CultureInfo to use as the current parsing culture.

`value` string

The string to convert.

`propertyType` Type

The property type that the converter will convert to.

Returns

object

A string that represents the converted value.

Remarks

Unlike other converters the Type property does not match the `propertyType` value. This allows us to reuse the same converter for infinite enum types.

Exceptions

NotSupportedException

The conversion cannot be performed.

Interface ICommandConverter

Namespace: SixLabors.ImageSharp.Web.Commands.Converters

Assembly: SixLabors.ImageSharp.Web.dll

Defines a contract for converting the value of a string into a different data type. Implementations should be stateless by design and also implement ICommandConverter<T>.

```
public interface ICommandConverter
```

Properties

Type

Gets the type this converter returns.

```
Type Type { get; }
```

Property Value

Type

Interface ICommandConverter<T>

Namespace: SixLabors.ImageSharp.Web.Commands.Converters

Assembly: SixLabors.ImageSharp.Web.dll

Defines a contract for converting the value of a string into a different data type. Implementations should be stateless by design and also implement ICommandConverter<T>.

```
public interface ICommandConverter<T> : ICommandConverter
```

Type Parameters

T

The type this converter returns.

Inherited Members
ICommandConverter.Type

Methods

ConvertFrom(CommandParser, CultureInfo, string?, Type)

Converts the given string to the type of this converter, using the specified culture information.

```
T? ConvertFrom(CommandParser parser, CultureInfo culture, string? value, Type propertyType)
```

Parameters

`parser` CommandParser

The command parser use for parting commands.

`culture` CultureInfo

The CultureInfo to use as the current parsing culture.

`value` string

The string to convert.

`propertyType` Type

The property type that the converter will convert to.

Returns

T

A string that represents the converted value.

Exceptions

NotSupportedException

The conversion cannot be performed.

Class IntegralNumberConverter<T>

Namespace: SixLabors.ImageSharp.Web.Commands.Converters

Assembly: SixLabors.ImageSharp.Web.dll

The generic converter for integral types.

```
public sealed class IntegralNumberConverter<T> : ICommandConverter<T>, ICommandConverter
where T : struct, IConvertible, IComparable<T>
```

Type Parameters

T

Inheritance

object ← IntegralNumberConverter<T>

Implements

ICommandConverter<T>, ICommandConverter

Inherited Members

object.Equals(object) , object.Equals(object, object) , object.GetHashCode() , object.GetType() , object.ReferenceEquals(object, object) , object.ToString()

Properties

Type

Gets the type this converter returns.

```
public Type Type { get; }
```

Property Value

Type

Methods

ConvertFrom(CommandParser, CultureInfo, string?, Type)

Converts the given string to the type of this converter, using the specified culture information.

```
public T ConvertFrom(CommandParser parser, CultureInfo culture, string? value,
Type propertyType)
```

Parameters

`parser` CommandParser

The command parser use for parting commands.

`culture` CultureInfo

The CultureInfo to use as the current parsing culture.

`value` string

The string to convert.

`propertyType` Type

The property type that the converter will convert to.

Returns

T

A string that represents the converted value.

Exceptions

NotSupportedException

The conversion cannot be performed.

Class ListConverter<T>

Namespace: SixLabors.ImageSharp.Web.Commands.Converters

Assembly: SixLabors.ImageSharp.Web.dll

Converts the value of a string to a generic list.

```
public sealed class ListConverter<T> : ICommandConverter<List<T>>, ICommandConverter
```

Type Parameters

T

The type of result to return.

Inheritance

object ← ListConverter<T>

Implements

ICommandConverter<List<T>>, ICommandConverter

Inherited Members

object.Equals(object) , object.Equals(object, object) , object.GetHashCode() , object.GetType() , object.ReferenceEquals(object, object) , object.ToString()

Properties

Type

Gets the type this converter returns.

```
public Type Type { get; }
```

Property Value

Type

Methods

ConvertFrom(CommandParser, CultureInfo, string?, Type)

Converts the given string to the type of this converter, using the specified culture information.

```
public List<T> ConvertFrom(CommandParser parser, CultureInfo culture, string? value,
Type propertyType)
```

Parameters

`parser` CommandParser

The command parser use for parting commands.

`culture` CultureInfo

The CultureInfo to use as the current parsing culture.

`value` string

The string to convert.

`propertyType` Type

The property type that the converter will convert to.

Returns

List<T>

A string that represents the converted value.

Exceptions

NotSupportedException

The conversion cannot be performed.

Class SimpleCommandConverter<T>

Namespace: SixLabors.ImageSharp.Web.Commands.Converters

Assembly: SixLabors.ImageSharp.Web.dll

The generic converter for simple types that implement IConvertible.

```
public sealed class SimpleCommandConverter<T> : ICommandConverter<T>, ICommandConverter
where T : IConvertible
```

Type Parameters

T

The type of object to convert to.

Inheritance

object ← SimpleCommandConverter<T>

Implements

ICommandConverter<T>, ICommandConverter

Inherited Members

object.Equals(object) , object.Equals(object, object) , object.GetHashCode() , object.GetType() , object.ReferenceEquals(object, object) , object.ToString()

Properties

Type

Gets the type this converter returns.

```
public Type Type { get; }
```

Property Value

Type

Methods

ConvertFrom(CommandParser, CultureInfo, string?, Type)

Converts the given string to the type of this converter, using the specified culture information.

```
public T? ConvertFrom(CommandParser parser, CultureInfo culture, string? value,
Type propertyType)
```

Parameters

`parser` CommandParser

The command parser use for parting commands.

`culture` CultureInfo

The CultureInfo to use as the current parsing culture.

`value` string

The string to convert.

`propertyType` Type

The property type that the converter will convert to.

Returns

T

A string that represents the converted value.

Exceptions

NotSupportedException

The conversion cannot be performed.

Namespace SixLabors.ImageSharp.Web.Middleware

Classes

ImageCommandContext
: Contains information about the current image request and parsed commands.

ImageProcessingContext
: Contains information about the current image request and processed image.

ImageSharpMiddleware
: Middleware for handling the processing of images via image requests.

ImageSharpMiddlewareOptions
: Configuration options for the ImageSharpMiddleware middleware.

Class ImageCommandContext

Namespace: SixLabors.ImageSharp.Web.Middleware

Assembly: SixLabors.ImageSharp.Web.dll

Contains information about the current image request and parsed commands.

```
public class ImageCommandContext
```

Inheritance

object ← ImageCommandContext

Inherited Members

object.Equals(object) , object.Equals(object, object) , object.GetHashCode() , object.GetType() , object.MemberwiseClone() , object.ReferenceEquals(object, object) , object.ToString()

Constructors

ImageCommandContext(HttpContext, CommandCollection, CommandParser, CultureInfo)

Initializes a new instance of the ImageCommandContext class.

```
public ImageCommandContext(HttpContext context, CommandCollection commands, CommandParser
parser, CultureInfo culture)
```

Parameters

`context` HttpContext

The current HTTP request context.

`commands` CommandCollection

The dictionary containing the collection of URI derived processing commands.

`parser` CommandParser

The command parser for parsing URI derived processing commands.

`culture` CultureInfo

The culture used to parse commands.

Properties

Commands

Gets the collection of URI derived processing commands.

```
public CommandCollection Commands { get; }
```

Property Value

CommandCollection

Context

Gets the current HTTP request context.

```
public HttpContext Context { get; }
```

Property Value

HttpContext

Culture

Gets the culture used for parsing commands.

```
public CultureInfo Culture { get; }
```

Property Value

CultureInfo

Parser

Gets the command parser for parsing URI derived processing commands.

```
public CommandParser Parser { get; }
```

Property Value

CommandParser

Class ImageProcessingContext

Namespace: SixLabors.ImageSharp.Web.Middleware

Assembly: SixLabors.ImageSharp.Web.dll

Contains information about the current image request and processed image.

```
public class ImageProcessingContext
```

Inheritance

object ← ImageProcessingContext

Inherited Members

object.Equals(object) , object.Equals(object, object) , object.GetHashCode() , object.GetType() , object.MemberwiseClone() , object.ReferenceEquals(object, object) , object.ToString()

Constructors

ImageProcessingContext(HttpContext, Stream, CommandCollection, string, string)

Initializes a new instance of the ImageProcessingContext class.

```
public ImageProcessingContext(HttpContext context, Stream stream, CommandCollection
commands, string contentType, string extension)
```

Parameters

`context` HttpContext

The current HTTP request context.

`stream` Stream

The stream containing the processed image bytes.

`commands` CommandCollection

The parsed collection of processing commands.

`contentType` string

The content type for the processed image.

`extension` string

The file extension for the processed image.

Properties

Commands

Gets the parsed collection of processing commands.

```
public CommandCollection Commands { get; }
```

Property Value

CommandCollection

ContentType

Gets the content type for the processed image.

```
public string ContentType { get; }
```

Property Value

string

Context

Gets the current HTTP request context.

```
public HttpContext Context { get; }
```

Property Value

HttpContext

Extension

Gets the file extension for the processed image.

```
public string Extension { get; }
```

Property Value

string

Stream

Gets the stream containing the processed image bytes.

```
public Stream Stream { get; }
```

Property Value

Stream

Class ImageSharpMiddleware

Namespace: SixLabors.ImageSharp.Web.Middleware

Assembly: SixLabors.ImageSharp.Web.dll

Middleware for handling the processing of images via image requests.

```
public class ImageSharpMiddleware
```

Inheritance

object ← ImageSharpMiddleware

Inherited Members

object.Equals(object) , object.Equals(object, object) , object.GetHashCode() , object.GetType() , object.MemberwiseClone() , object.ReferenceEquals(object, object) , object.ToString()

Constructors

ImageSharpMiddleware(RequestDelegate, IOptions<ImageSharpMiddlewareOptions>, ILoggerFactory, IRequestParser, IEnumerable<IImageProvider>, IEnumerable<IImageWebProcessor>, IImageCache, ICacheKey, ICacheHash, CommandParser, FormatUtilities, AsyncKeyReaderWriterLock<string>, RequestAuthorizationUtilities)

Initializes a new instance of the ImageSharpMiddleware class.

```
public ImageSharpMiddleware(RequestDelegate next, IOptions<ImageSharpMiddlewareOptions>
options, ILoggerFactory loggerFactory, IRequestParser requestParser,
IEnumerable<IImageProvider> resolvers, IEnumerable<IImageWebProcessor> processors,
IImageCache cache, ICacheKey cacheKey, ICacheHash cacheHash, CommandParser commandParser,
FormatUtilities formatUtilities, AsyncKeyReaderWriterLock<string> asyncKeyLock,
RequestAuthorizationUtilities requestAuthorizationUtilities)
```

Parameters

`next` RequestDelegate

The next middleware in the pipeline.

`options` IOptions<ImageSharpMiddlewareOptions>

The middleware configuration options.

`loggerFactory` ILoggerFactory

An ILoggerFactory instance used to create loggers.

`requestParser` IRequestParser

An IRequestParser instance used to parse image requests for commands.

`resolvers` IEnumerable<IImageProvider>

A collection of IImageProvider instances used to resolve images.

`processors` IEnumerable<IImageWebProcessor>

A collection of IImageWebProcessor instances used to process images.

`cache` IImageCache

An IImageCache instance used for caching images.

`cacheKey` ICacheKey

An ICacheKey instance used for creating cache keys.

`cacheHash` ICacheHash

An ICacheHashinstance used for calculating cached file names.

`commandParser` CommandParser

The command parser.

`formatUtilities` FormatUtilities

Contains various format helper methods based on the current configuration.

`asyncKeyLock` AsyncKeyReaderWriterLock<string>

The async key lock.

`requestAuthorizationUtilities` RequestAuthorizationUtilities

Contains helpers that allow authorization of image requests.

Methods

Invoke(HttpContext)

Performs operations upon the current request.

```
public Task Invoke(HttpContext httpContext)
```

Parameters

`httpContext` HttpContext

The current HTTP request context.

Returns

Task

The Task.

Class ImageSharpMiddlewareOptions

Namespace: SixLabors.ImageSharp.Web.Middleware

Assembly: SixLabors.ImageSharp.Web.dll

Configuration options for the ImageSharpMiddleware middleware.

```
public class ImageSharpMiddlewareOptions
```

Inheritance

object ← ImageSharpMiddlewareOptions

Inherited Members

object.Equals(object) , object.Equals(object, object) , object.GetHashCode() , object.GetType() , object.MemberwiseClone() , object.ReferenceEquals(object, object) , object.ToString()

Properties

BrowserMaxAge

Gets or sets the duration to store images in the browser cache. If an image provider provides a Max-Age for a source image then that will override this value.

Defaults to 7 days.

```
public TimeSpan BrowserMaxAge { get; set; }
```

Property Value

TimeSpan

CacheHashLength

Gets or sets the length of the filename to use (minus the extension) when storing images in the image cache. Defaults to 12 characters.

```
public uint CacheHashLength { get; set; }
```

Property Value

uint

CacheMaxAge

Gets or sets the duration to store images in the image cache.

Defaults to 365 days.

```
public TimeSpan CacheMaxAge { get; set; }
```

Property Value

TimeSpan

Configuration

Gets or sets the base library configuration.

```
public Configuration Configuration { get; set; }
```

Property Value

Configuration

HMACSecretKey

Gets or sets the secret key for Hash-based Message Authentication Code (HMAC) encryption.

```
public byte[] HMACSecretKey { get; set; }
```

Property Value

byte[]

Remarks

The key can be any length. However, the recommended size is at least 64 bytes. If the length is zero then no authentication is performed.

MemoryStreamManager

Gets or sets the recyclable memorystream manager used for managing pooled stream buffers independently from image buffer pooling.

```
public RecyclableMemoryStreamManager MemoryStreamManager { get; set; }
```

Property Value

RecyclableMemoryStreamManager

OnBeforeLoadAsync

Gets or sets the method that can be used to used to augment decoder options. This is called before the image is decoded and loaded for processing.

```
public Func<ImageCommandContext, Configuration, Task<DecoderOptions?>> OnBeforeLoadAsync {
get; set; }
```

Property Value

Func<ImageCommandContext, Configuration, Task<DecoderOptions>>

OnBeforeSaveAsync

Gets or sets the additional method that can be used for final manipulation before the image is saved. This is called after image has been processed, but before the image has been saved to the

output stream for caching. This can be used to alter the metadata of the resultant image.

```
public Func<FormattedImage, Task> OnBeforeSaveAsync { get; set; }
```

Property Value

Func<FormattedImage, Task>

OnComputeHMAC

Gets or sets the method used to compute a Hash-based Message Authentication Code (HMAC) for request authentication. Defaults to ComputeHMACSHA256(string, byte[]) using an invariant lowercase relative Uri generated using BuildRelative(CaseHandling, PathString, PathString, QueryString).

```
public Func<ImageCommandContext, byte[], string> OnComputeHMAC { get; set; }
```

Property Value

Func<ImageCommandContext, byte[], string>

OnParseCommandsAsync

Gets or sets the additional command parsing method that can be used to used to augment commands. This is called once the commands have been gathered and before an IImageProvider has been assigned.

```
public Func<ImageCommandContext, Task> OnParseCommandsAsync { get; set; }
```

Property Value

Func<ImageCommandContext, Task>

OnPrepareResponseAsync

Gets or sets the additional response method. This is called after the status code and headers have been set, but before the body has been written. This can be used to add or change the response headers.

```
public Func<HttpContext, Task> OnPrepareResponseAsync { get; set; }
```

Property Value

Func<HttpContext, Task>

OnProcessedAsync

Gets or sets the additional processing method. This is called after image has been processed, but before the result has been cached. This can be used to further optimize the resultant image.

```
public Func<ImageProcessingContext, Task> OnProcessedAsync { get; set; }
```

Property Value

Func<ImageProcessingContext, Task>

UseInvariantParsingCulture

Gets or sets a value indicating whether to use culture-independent (invariant) conversion when converting commands. If set to false the CommandParser will use the CurrentCulture.

```
public bool UseInvariantParsingCulture { get; set; }
```

Property Value

bool

Namespace SixLabors.ImageSharp.Web.Processors

Classes

AutoOrientWebProcessor
: Allows the auto-orientation to ensure that EXIF defined orientation is reflected in the final image.

BackgroundColorWebProcessor
: Allows setting of the background color.

FormatWebProcessor
: Allows the changing of image formats.

QualityWebProcessor
: Allows the setting of quality for the jpeg and webp image format.

ResizeWebProcessor
: Allows the resizing of images.

Interfaces

IImageWebProcessor
: Specifies the contract for processing images using a querystring URI API.

Class AutoOrientWebProcessor

Namespace: SixLabors.ImageSharp.Web.Processors

Assembly: SixLabors.ImageSharp.Web.dll

Allows the auto-orientation to ensure that EXIF defined orientation is reflected in the final image.

```
public class AutoOrientWebProcessor : IImageWebProcessor
```

Inheritance

object ← AutoOrientWebProcessor

Implements

IImageWebProcessor

Inherited Members

object.Equals(object) , object.Equals(object, object) , object.GetHashCode() , object.GetType() , object.MemberwiseClone() , object.ReferenceEquals(object, object) , object.ToString()

Fields

AutoOrient

The command for changing the orientation according to the EXIF information.

```
public const string AutoOrient = "autoorient"
```

Field Value

string

Properties

Commands

Gets the collection of recognized command keys.

```
public IEnumerable<string> Commands { get; }
```

Property Value

IEnumerable<string>

Methods

Process(FormattedImage, ILogger, CommandCollection, CommandParser, CultureInfo)

Processes the image based on the given commands.

```
public FormattedImage Process(FormattedImage image, ILogger logger, CommandCollection
commands, CommandParser parser, CultureInfo culture)
```

Parameters

`image` FormattedImage

The image to process.

`logger` ILogger

The type used for performing logging.

`commands` CommandCollection

The ordered collection containing the processing commands.

`parser` CommandParser

The command parser use for parting commands.

`culture` CultureInfo

The CultureInfo to use as the current parsing culture.

Returns

FormattedImage

The FormattedImage.

RequiresTrueColorPixelFormat(CommandCollection, CommandParser, CultureInfo)

Returns a value indicating whether the image to be processed should be decoded using a 32 bit True Color pixel format - 8 bits per color component plus an 8 bit alpha channel https://en.wikipedia.org/wiki/Color_depth#True_color_(24-bit).

This method is used to determine whether optimizations can be enabled to reduce memory consumption during processing.

```
public bool RequiresTrueColorPixelFormat(CommandCollection commands, CommandParser parser,
CultureInfo culture)
```

Parameters

`commands` CommandCollection

The ordered collection containing the processing commands.

`parser` CommandParser

The command parser use for parting commands.

`culture` CultureInfo

The CultureInfo to use as the current parsing culture.

Returns

bool

The bool indicating whether a 32 bit True Color pixel format is required.

Class BackgroundColorWebProcessor

Namespace: SixLabors.ImageSharp.Web.Processors

Assembly: SixLabors.ImageSharp.Web.dll

Allows setting of the background color.

```
public class BackgroundColorWebProcessor : IImageWebProcessor
```

Inheritance

object ← BackgroundColorWebProcessor

Implements

IImageWebProcessor

Inherited Members

object.Equals(object) , object.Equals(object, object) , object.GetHashCode() , object.GetType() , object.MemberwiseClone() , object.ReferenceEquals(object, object) , object.ToString()

Fields

Color

The command for changing the background color.

```
public const string Color = "bgcolor"
```

Field Value

string

Properties

Commands

Gets the collection of recognized command keys.

```
public IEnumerable<string> Commands { get; }
```

Property Value

IEnumerable<string>

Methods

Process(FormattedImage, ILogger, CommandCollection, CommandParser, CultureInfo)

Processes the image based on the given commands.

```
public FormattedImage Process(FormattedImage image, ILogger logger, CommandCollection
commands, CommandParser parser, CultureInfo culture)
```

Parameters

`image` FormattedImage

The image to process.

`logger` ILogger

The type used for performing logging.

`commands` CommandCollection

The ordered collection containing the processing commands.

`parser` CommandParser

The command parser use for parting commands.

`culture` CultureInfo

The CultureInfo to use as the current parsing culture.

Returns

FormattedImage

The FormattedImage.

RequiresTrueColorPixelFormat(CommandCollection, CommandParser, CultureInfo)

Returns a value indicating whether the image to be processed should be decoded using a 32 bit True Color pixel format - 8 bits per color component plus an 8 bit alpha channel https://en.wikipedia.org/wiki/Color_depth#True_color_(24-bit).

This method is used to determine whether optimizations can be enabled to reduce memory consumption during processing.

```
public bool RequiresTrueColorPixelFormat(CommandCollection commands, CommandParser parser,
CultureInfo culture)
```

Parameters

`commands` CommandCollection

The ordered collection containing the processing commands.

`parser` CommandParser

The command parser use for parting commands.

`culture` CultureInfo

The CultureInfo to use as the current parsing culture.

Returns

bool

The bool indicating whether a 32 bit True Color pixel format is required.

Class FormatWebProcessor

Namespace: SixLabors.ImageSharp.Web.Processors

Assembly: SixLabors.ImageSharp.Web.dll

Allows the changing of image formats.

```
public class FormatWebProcessor : IImageWebProcessor
```

Inheritance

object ← FormatWebProcessor

Implements

IImageWebProcessor

Inherited Members

object.Equals(object) , object.Equals(object, object) , object.GetHashCode() , object.GetType() , object.MemberwiseClone() , object.ReferenceEquals(object, object) , object.ToString()

Constructors

FormatWebProcessor(IOptions<ImageSharpMiddlewareOptions>)

Initializes a new instance of the FormatWebProcessor class.

```
public FormatWebProcessor(IOptions<ImageSharpMiddlewareOptions> options)
```

Parameters

`options` IOptions<ImageSharpMiddlewareOptions>

The middleware configuration options.

Fields

Format

The command constant for format.

```
public const string Format = "format"
```

Field Value

string

Properties

Commands

Gets the collection of recognized command keys.

```
public IEnumerable<string> Commands { get; }
```

Property Value

IEnumerable<string>

Methods

Process(FormattedImage, ILogger, CommandCollection, CommandParser, CultureInfo)

Processes the image based on the given commands.

```
public FormattedImage Process(FormattedImage image, ILogger logger, CommandCollection
commands, CommandParser parser, CultureInfo culture)
```

Parameters

`image` FormattedImage

The image to process.

`logger` ILogger

The type used for performing logging.

`commands` CommandCollection

The ordered collection containing the processing commands.

`parser` CommandParser

The command parser use for parting commands.

`culture` CultureInfo

The CultureInfo to use as the current parsing culture.

Returns

FormattedImage

The FormattedImage.

RequiresTrueColorPixelFormat(CommandCollection, CommandParser, CultureInfo)

Returns a value indicating whether the image to be processed should be decoded using a 32 bit True Color pixel format - 8 bits per color component plus an 8 bit alpha channel https://en.wikipedia.org/wiki/Color_depth#True_color_(24-bit).

This method is used to determine whether optimizations can be enabled to reduce memory consumption during processing.

```
public bool RequiresTrueColorPixelFormat(CommandCollection commands, CommandParser parser,
CultureInfo culture)
```

Parameters

`commands` CommandCollection

The ordered collection containing the processing commands.

`parser` CommandParser

The command parser use for parting commands.

`culture` CultureInfo

The CultureInfo to use as the current parsing culture.

Returns

bool

The bool indicating whether a 32 bit True Color pixel format is required.

Interface IImageWebProcessor

Namespace: SixLabors.ImageSharp.Web.Processors

Assembly: SixLabors.ImageSharp.Web.dll

Specifies the contract for processing images using a querystring URI API.

```
public interface IImageWebProcessor
```

Properties

Commands

Gets the collection of recognized command keys.

```
IEnumerable<string> Commands { get; }
```

Property Value

IEnumerable<string>

Methods

Process(FormattedImage, ILogger, CommandCollection, CommandParser, CultureInfo)

Processes the image based on the given commands.

```
FormattedImage Process(FormattedImage image, ILogger logger, CommandCollection commands,
CommandParser parser, CultureInfo culture)
```

Parameters

`image` FormattedImage

The image to process.

`logger` ILogger

The type used for performing logging.

`commands` CommandCollection

The ordered collection containing the processing commands.

`parser` CommandParser

The command parser use for parting commands.

`culture` CultureInfo

The CultureInfo to use as the current parsing culture.

Returns

FormattedImage

The FormattedImage.

RequiresTrueColorPixelFormat(CommandCollection, CommandParser, CultureInfo)

Returns a value indicating whether the image to be processed should be decoded using a 32 bit True Color pixel format - 8 bits per color component plus an 8 bit alpha channel https://en.wikipedia.org/wiki/Color_depth#True_color_(24-bit).

This method is used to determine whether optimizations can be enabled to reduce memory consumption during processing.

```
bool RequiresTrueColorPixelFormat(CommandCollection commands, CommandParser parser,
CultureInfo culture)
```

Parameters

`commands` CommandCollection

The ordered collection containing the processing commands.

`parser` CommandParser

The command parser use for parting commands.

`culture` CultureInfo

The CultureInfo to use as the current parsing culture.

Returns

bool

The bool indicating whether a 32 bit True Color pixel format is required.

Class QualityWebProcessor

Namespace: SixLabors.ImageSharp.Web.Processors

Assembly: SixLabors.ImageSharp.Web.dll

Allows the setting of quality for the jpeg and webp image format.

```
public class QualityWebProcessor : IImageWebProcessor
```

Inheritance

object ← QualityWebProcessor

Implements

IImageWebProcessor

Inherited Members

object.Equals(object) , object.Equals(object, object) , object.GetHashCode() , object.GetType() , object.MemberwiseClone() , object.ReferenceEquals(object, object) , object.ToString()

Fields

Quality

The command constant for quality.

```
public const string Quality = "quality"
```

Field Value

string

Properties

Commands

Gets the collection of recognized command keys.

```
public IEnumerable<string> Commands { get; }
```

Property Value

IEnumerable<string>

Methods

Process(FormattedImage, ILogger, CommandCollection, CommandParser, CultureInfo)

Processes the image based on the given commands.

```
public FormattedImage Process(FormattedImage image, ILogger logger, CommandCollection
commands, CommandParser parser, CultureInfo culture)
```

Parameters

`image` FormattedImage

The image to process.

`logger` ILogger

The type used for performing logging.

`commands` CommandCollection

The ordered collection containing the processing commands.

`parser` CommandParser

The command parser use for parting commands.

`culture` CultureInfo

The CultureInfo to use as the current parsing culture.

Returns

FormattedImage

The FormattedImage.

RequiresTrueColorPixelFormat(CommandCollection, CommandParser, CultureInfo)

Returns a value indicating whether the image to be processed should be decoded using a 32 bit True Color pixel format - 8 bits per color component plus an 8 bit alpha channel https://en.wikipedia.org/wiki/Color_depth#True_color_(24-bit).

This method is used to determine whether optimizations can be enabled to reduce memory consumption during processing.

```
public bool RequiresTrueColorPixelFormat(CommandCollection commands, CommandParser parser,
CultureInfo culture)
```

Parameters

`commands` CommandCollection

The ordered collection containing the processing commands.

`parser` CommandParser

The command parser use for parting commands.

`culture` CultureInfo

The CultureInfo to use as the current parsing culture.

Returns

bool

The bool indicating whether a 32 bit True Color pixel format is required.

Class ResizeWebProcessor

Namespace: SixLabors.ImageSharp.Web.Processors

Assembly: SixLabors.ImageSharp.Web.dll

Allows the resizing of images.

```
public class ResizeWebProcessor : IImageWebProcessor
```

Inheritance

object ← ResizeWebProcessor

Implements

IImageWebProcessor

Inherited Members

object.Equals(object) , object.Equals(object, object) , object.GetHashCode() , object.GetType() , object.MemberwiseClone() , object.ReferenceEquals(object, object) , object.ToString()

Fields

Anchor

The command constant for the resize anchor position.

```
public const string Anchor = "ranchor"
```

Field Value

string

Color

The command constant for the resize padding background color.

```
public const string Color = "rcolor"
```

Field Value

string

Compand

The command constant for the resize compand mode.

```
public const string Compand = "compand"
```

Field Value

string

Height

The command constant for the resize height.

```
public const string Height = "height"
```

Field Value

string

Mode

The command constant for the resize mode.

```
public const string Mode = "rmode"
```

Field Value

string

Orient

The command constant for the resize orientation handling mode.

```
public const string Orient = "orient"
```

Field Value

string

Sampler

The command constant for the resize sampler.

```
public const string Sampler = "rsampler"
```

Field Value

string

Width

The command constant for the resize width.

```
public const string Width = "width"
```

Field Value

string

Xy

The command constant for the resize focal point coordinates.

```
public const string Xy = "rxy"
```

Field Value

string

Properties

Commands

Gets the collection of recognized command keys.

```
public IEnumerable<string> Commands { get; }
```

Property Value

IEnumerable<string>

Methods

Process(FormattedImage, ILogger, CommandCollection, CommandParser, CultureInfo)

Processes the image based on the given commands.

```
public FormattedImage Process(FormattedImage image, ILogger logger, CommandCollection
commands, CommandParser parser, CultureInfo culture)
```

Parameters

`image` FormattedImage

The image to process.

`logger` ILogger

The type used for performing logging.

`commands` CommandCollection

The ordered collection containing the processing commands.

`parser` CommandParser

The command parser use for parting commands.

`culture` CultureInfo

The CultureInfo to use as the current parsing culture.

Returns

FormattedImage

The FormattedImage.

RequiresTrueColorPixelFormat(CommandCollection, CommandParser, CultureInfo)

Returns a value indicating whether the image to be processed should be decoded using a 32 bit True Color pixel format - 8 bits per color component plus an 8 bit alpha channel https://en.wikipedia.org/wiki/Color_depth#True_color_(24-bit).

This method is used to determine whether optimizations can be enabled to reduce memory consumption during processing.

```
public bool RequiresTrueColorPixelFormat(CommandCollection commands, CommandParser parser,
CultureInfo culture)
```

Parameters

`commands` CommandCollection

The ordered collection containing the processing commands.

`parser` CommandParser

The command parser use for parting commands.

`culture` CultureInfo

The CultureInfo to use as the current parsing culture.

Returns

bool

The bool indicating whether a 32 bit True Color pixel format is required.

Namespace SixLabors.ImageSharp.Web.Providers

Classes

FileProviderImageProvider
: Returns images from an IFileProvider abstraction.

PhysicalFileSystemProvider
: Returns images stored in the local physical file system.

PhysicalFileSystemProviderOptions
: Configuration options for the PhysicalFileSystemProvider.

WebRootImageProvider
: Returns images from the web root file provider.

Interfaces

IImageProvider
: Specifies the contract for returning images from different locations.

Enums

ProcessingBehavior
: Enumerates the possible processing behaviors.

Class FileProviderImageProvider

Namespace: SixLabors.ImageSharp.Web.Providers

Assembly: SixLabors.ImageSharp.Web.dll

Returns images from an IFileProvider abstraction.

```
public abstract class FileProviderImageProvider : IImageProvider
```

Inheritance

object ← FileProviderImageProvider

Implements

IImageProvider

Derived

PhysicalFileSystemProvider, WebRootImageProvider

Inherited Members

object.Equals(object) , object.Equals(object, object) , object.GetHashCode() , object.GetType() , object.MemberwiseClone() , object.ReferenceEquals(object, object) , object.ToString()

Constructors

FileProviderImageProvider(IFileProvider, ProcessingBehavior, FormatUtilities)

Initializes a new instance of the FileProviderImageProvider class.

```
protected FileProviderImageProvider(IFileProvider fileProvider, ProcessingBehavior
processingBehavior, FormatUtilities formatUtilities)
```

Parameters

`fileProvider` IFileProvider

The file provider.

`processingBehavior` ProcessingBehavior

The processing behavior.

`formatUtilities` FormatUtilities

Contains various format helper methods based on the current configuration.

Properties

Match

Gets or sets the method used by the resolver to identify itself as the correct resolver to use.

```
public virtual Func<HttpContext, bool> Match { get; set; }
```

Property Value

Func<HttpContext, bool>

ProcessingBehavior

Gets the processing behavior.

```
public ProcessingBehavior ProcessingBehavior { get; }
```

Property Value

ProcessingBehavior

Methods

GetAsync(HttpContext)

Gets the image resolver associated with the context.

```
public Task<IImageResolver?> GetAsync(HttpContext context)
```

Parameters

`context` HttpContext

The current HTTP request context.

Returns

Task<IImageResolver>

The IImageResolver.

IsValidRequest(HttpContext)

Gets a value indicating whether the current request passes sanitizing rules.

```
public virtual bool IsValidRequest(HttpContext context)
```

Parameters

`context` HttpContext

The current HTTP request context.

Returns

bool

The bool

Interface IImageProvider

Namespace: SixLabors.ImageSharp.Web.Providers

Assembly: SixLabors.ImageSharp.Web.dll

Specifies the contract for returning images from different locations.

```
public interface IImageProvider
```

Properties

Match

Gets or sets the method used by the resolver to identify itself as the correct resolver to use.

```
Func<HttpContext, bool> Match { get; set; }
```

Property Value

Func<HttpContext, bool>

ProcessingBehavior

Gets the processing behavior.

```
ProcessingBehavior ProcessingBehavior { get; }
```

Property Value

ProcessingBehavior

Methods

GetAsync(HttpContext)

Gets the image resolver associated with the context.

```
Task<IImageResolver?> GetAsync(HttpContext context)
```

Parameters

`context` HttpContext

The current HTTP request context.

Returns

Task<IImageResolver>

The IImageResolver.

IsValidRequest(HttpContext)

Gets a value indicating whether the current request passes sanitizing rules.

```
bool IsValidRequest(HttpContext context)
```

Parameters

`context` HttpContext

The current HTTP request context.

Returns

bool

The bool

Class PhysicalFileSystemProvider

Namespace: SixLabors.ImageSharp.Web.Providers

Assembly: SixLabors.ImageSharp.Web.dll

Returns images stored in the local physical file system.

```
public sealed class PhysicalFileSystemProvider : FileProviderImageProvider, IImageProvider
```

Inheritance

object ← FileProviderImageProvider ← PhysicalFileSystemProvider

Implements

IImageProvider

Inherited Members

FileProviderImageProvider.ProcessingBehavior , FileProviderImageProvider.Match , FileProviderImageProvider.IsValidRequest(HttpContext) , FileProviderImageProvider.GetAsync(HttpContext) , object.Equals(object) , object.Equals(object, object) , object.GetHashCode() , object.GetType() , object.ReferenceEquals(object, object) , object.ToString()

Constructors

PhysicalFileSystemProvider(IOptions<PhysicalFileSystemProviderOptions>, IWebHostEnvironment, FormatUtilities)

Initializes a new instance of the PhysicalFileSystemProvider class.

```
public PhysicalFileSystemProvider(IOptions<PhysicalFileSystemProviderOptions> options,
IWebHostEnvironment environment, FormatUtilities formatUtilities)
```

Parameters

`options` IOptions<PhysicalFileSystemProviderOptions>

The provider configuration options.

`environment` IWebHostEnvironment

The environment used by this middleware.

`formatUtilities` FormatUtilities

Contains various format helper methods based on the current configuration.

Class PhysicalFileSystemProviderOptions

Namespace: SixLabors.ImageSharp.Web.Providers

Assembly: SixLabors.ImageSharp.Web.dll

Configuration options for the PhysicalFileSystemProvider.

```
public class PhysicalFileSystemProviderOptions
```

Inheritance

object ← PhysicalFileSystemProviderOptions

Inherited Members

object.Equals(object) , object.Equals(object, object) , object.GetHashCode() , object.GetType() , object.MemberwiseClone() , object.ReferenceEquals(object, object) , object.ToString()

Properties

ProcessingBehavior

Gets or sets the processing behavior. Defaults to CommandOnly.

```
public ProcessingBehavior ProcessingBehavior { get; set; }
```

Property Value

ProcessingBehavior

ProviderRootPath

Gets or sets the optional provider root folder path.

This value can be null, a fully qualified absolute path, or a path relative to the directory that contains the application content files.

If not set, this will default to the directory that contains the web-servable application content files; commonly 'wwwroot'.

```
public string? ProviderRootPath { get; set; }
```

Property Value

string

Enum ProcessingBehavior

Namespace: SixLabors.ImageSharp.Web.Providers

Assembly: SixLabors.ImageSharp.Web.dll

Enumerates the possible processing behaviors.

```
public enum ProcessingBehavior
```

Fields

`All = 1`

The image will always be processed.

`CommandOnly = 0`

The image will be processed only when commands are supplied.

Class WebRootImageProvider

Namespace: SixLabors.ImageSharp.Web.Providers

Assembly: SixLabors.ImageSharp.Web.dll

Returns images from the web root file provider.

```
public sealed class WebRootImageProvider : FileProviderImageProvider, IImageProvider
```

Inheritance

object ← FileProviderImageProvider ← WebRootImageProvider

Implements

IImageProvider

Inherited Members

FileProviderImageProvider.ProcessingBehavior , FileProviderImageProvider.Match , FileProviderImageProvider.IsValidRequest(HttpContext) , FileProviderImageProvider.GetAsync(HttpContext) , object.Equals(object) , object.Equals(object, object) , object.GetHashCode() , object.GetType() , object.ReferenceEquals(object, object) , object.ToString()

Constructors

WebRootImageProvider(IWebHostEnvironment, FormatUtilities)

Initializes a new instance of the WebRootImageProvider class.

```
public WebRootImageProvider(IWebHostEnvironment environment,
FormatUtilities formatUtilities)
```

Parameters

`environment` IWebHostEnvironment

The web hosting environment.

`formatUtilities` FormatUtilities

Contains various format helper methods based on the current configuration.

Namespace SixLabors.ImageSharp.Web.Resolvers

Classes

FileProviderImageResolver
: Provides means to manage image buffers from an IFileInfo instance.

PhysicalFileSystemCacheResolver
: Provides means to manage image buffers within the PhysicalFileSystemCache.

Interfaces

IImageCacheResolver
: Specifies the contract for resolving image buffers from different cached locations.

IImageResolver
: Specifies the contract for resolving image buffers from different locations.

Class FileProviderImageResolver

Namespace: SixLabors.ImageSharp.Web.Resolvers

Assembly: SixLabors.ImageSharp.Web.dll

Provides means to manage image buffers from an IFileInfo instance.

```
public sealed class FileProviderImageResolver : IImageResolver
```

Inheritance

object ← FileProviderImageResolver

Implements

IImageResolver

Inherited Members

object.Equals(object) , object.Equals(object, object) , object.GetHashCode() , object.GetType() , object.ReferenceEquals(object, object) , object.ToString()

Constructors

FileProviderImageResolver(IFileInfo)

Initializes a new instance of the FileProviderImageResolver class.

```
public FileProviderImageResolver(IFileInfo fileInfo)
```

Parameters

`fileInfo` IFileInfo

The file info.

Methods

GetMetaDataAsync()

Asynchronously gets metadata associated with this image.

```
public Task<ImageMetadata> GetMetaDataAsync()
```

Returns

Task<ImageMetadata>

The ImageMetadata.

OpenReadAsync()

Asynchronously gets the input image stream.

```
public Task<Stream> OpenReadAsync()
```

Returns

Task<Stream>

The Task<TResult>.

Interface IImageCacheResolver

Namespace: SixLabors.ImageSharp.Web.Resolvers

Assembly: SixLabors.ImageSharp.Web.dll

Specifies the contract for resolving image buffers from different cached locations.

```
public interface IImageCacheResolver
```

Methods

GetMetaDataAsync()

Asynchronously gets metadata associated with this image.

```
Task<ImageCacheMetadata> GetMetaDataAsync()
```

Returns

Task<ImageCacheMetadata>

The ImageCacheMetadata.

OpenReadAsync()

Asynchronously gets the input image stream.

```
Task<Stream> OpenReadAsync()
```

Returns

Task<Stream>

The Task<TResult>.

Interface IImageResolver

Namespace: SixLabors.ImageSharp.Web.Resolvers

Assembly: SixLabors.ImageSharp.Web.dll

Specifies the contract for resolving image buffers from different locations.

```
public interface IImageResolver
```

Methods

GetMetaDataAsync()

Asynchronously gets metadata associated with this image.

```
Task<ImageMetadata> GetMetaDataAsync()
```

Returns

Task<ImageMetadata>

The ImageMetadata.

OpenReadAsync()

Asynchronously gets the input image stream.

```
Task<Stream> OpenReadAsync()
```

Returns

Task<Stream>

The Task<TResult>.

Class PhysicalFileSystemCacheResolver

Namespace: SixLabors.ImageSharp.Web.Resolvers

Assembly: SixLabors.ImageSharp.Web.dll

Provides means to manage image buffers within the PhysicalFileSystemCache.

```
public class PhysicalFileSystemCacheResolver : IImageCacheResolver
```

Inheritance

object ← PhysicalFileSystemCacheResolver

Implements

IImageCacheResolver

Inherited Members

object.Equals(object) , object.Equals(object, object) , object.GetHashCode() , object.GetType() , object.MemberwiseClone() , object.ReferenceEquals(object, object) , object.ToString()

Constructors

PhysicalFileSystemCacheResolver(FileInfo, FormatUtilities)

Initializes a new instance of the PhysicalFileSystemCacheResolver class.

```
public PhysicalFileSystemCacheResolver(FileInfo metaFileInfo,
FormatUtilities formatUtilities)
```

Parameters

`metaFileInfo` FileInfo

The cached metadata file info.

`formatUtilities` FormatUtilities

Contains various format helper methods based on the current configuration.

Methods

GetMetaDataAsync()

Asynchronously gets metadata associated with this image.

```
public Task<ImageCacheMetadata> GetMetaDataAsync()
```

Returns

Task<ImageCacheMetadata>

The ImageCacheMetadata.

OpenReadAsync()

Asynchronously gets the input image stream.

```
public Task<Stream> OpenReadAsync()
```

Returns

Task<Stream>

The Task<TResult>.

Namespace SixLabors.ImageSharp.Web.Synchronization

Classes

AsyncKeyLock<TKey>
: Extension of the AsyncLock that enables fine-grained locking on a given key. Concurrent lock requests using different keys can execute simultaneously, while requests to lock using the same key will be forced to wait. This object is thread-safe and internally uses a pooling mechanism to minimize allocation of new locks.

AsyncKeyReaderWriterLock<TKey>
: Extension of the AsyncReaderWriterLock that enables fine-grained locking on a given key. Concurrent write lock requests using different keys can execute simultaneously, while requests to lock using the same key will be forced to wait. This object is thread-safe and internally uses a pooling mechanism to minimize allocation of new locks.

AsyncLock
: An asynchronous locker that uses an IDisposable pattern for releasing the lock.

AsyncReaderWriterLock
: An asynchronous locker that provides read and write locking policies.

  This is based on the following blog post: https://devblogs.microsoft.com/pfxteam/building-async-coordination-primitives-part-7-asyncreaderwriterlock/

RefCountedConcurrentDictionary<TKey, TValue>
: Represents a thread-safe collection of reference-counted key/value pairs that can be accessed by multiple threads concurrently. Values that don't yet exist are automatically created using a caller supplied value factory method, and when their final refcount is released they are removed from the dictionary.

Class AsyncKeyLock<TKey>

Namespace: SixLabors.ImageSharp.Web.Synchronization

Assembly: SixLabors.ImageSharp.Web.dll

Extension of the AsyncLock that enables fine-grained locking on a given key. Concurrent lock requests using different keys can execute simultaneously, while requests to lock using the same key will be forced to wait. This object is thread-safe and internally uses a pooling mechanism to minimize allocation of new locks.

```
public sealed class AsyncKeyLock<TKey> : IDisposable where TKey : notnull
```

Type Parameters

TKey

The type of the key.

Inheritance

object ← AsyncKeyLock<TKey>

Implements

IDisposable

Inherited Members

object.Equals(object) , object.Equals(object, object) , object.GetHashCode() , object.GetType() , object.ReferenceEquals(object, object) , object.ToString()

Constructors

AsyncKeyLock(int)

Initializes a new instance of the AsyncKeyLock<TKey> class.

```
public AsyncKeyLock(int maxPoolSize = 64)
```

Parameters

`maxPoolSize` int

The maximum number of locks that should be pooled for reuse.

Methods

Dispose()

Releases all resources used by the current instance of the AsyncKeyLock<TKey> class.

```
public void Dispose()
```

LockAsync(TKey)

Locks the current thread asynchronously.

```
public Task<IDisposable> LockAsync(TKey key)
```

Parameters

`key` TKey

The key identifying the specific object to lock against.

Returns

Task<IDisposable>

The IDisposable that will release the lock.

Class AsyncKeyReaderWriterLock<TKey>

Namespace: SixLabors.ImageSharp.Web.Synchronization

Assembly: SixLabors.ImageSharp.Web.dll

Extension of the AsyncReaderWriterLock that enables fine-grained locking on a given key. Concurrent write lock requests using different keys can execute simultaneously, while requests to lock using the same key will be forced to wait. This object is thread-safe and internally uses a pooling mechanism to minimize allocation of new locks.

```
public class AsyncKeyReaderWriterLock<TKey> where TKey : notnull
```

Type Parameters

TKey

The type of the key.

Inheritance

object ← AsyncKeyReaderWriterLock<TKey>

Inherited Members

object.Equals(object) , object.Equals(object, object) , object.GetHashCode() , object.GetType() , object.MemberwiseClone() , object.ReferenceEquals(object, object) , object.ToString()

Constructors

AsyncKeyReaderWriterLock(int)

Initializes a new instance of the AsyncKeyReaderWriterLock<TKey> class.

```
public AsyncKeyReaderWriterLock(int maxPoolSize = 64)
```

Parameters

maxPoolSize int

The maximum number of locks that should be pooled for reuse.

Methods

ReaderLockAsync(TKey)

Locks the current thread in read mode asynchronously.

```
public Task<IDisposable> ReaderLockAsync(TKey key)
```

Parameters

`key` TKey

The key identifying the specific object to lock against.

Returns

Task<IDisposable>

The IDisposable that will release the lock.

WriterLockAsync(TKey)

Locks the current thread in write mode asynchronously.

```
public Task<IDisposable> WriterLockAsync(TKey key)
```

Parameters

`key` TKey

The key identifying the specific object to lock against.

Returns

Task<IDisposable>

The IDisposable that will release the lock.

Class AsyncLock

Namespace: SixLabors.ImageSharp.Web.Synchronization

Assembly: SixLabors.ImageSharp.Web.dll

An asynchronous locker that uses an IDisposable pattern for releasing the lock.

```
public sealed class AsyncLock : IDisposable
```

Inheritance

object ← AsyncLock

Implements

IDisposable

Inherited Members

object.Equals(object) , object.Equals(object, object) , object.GetHashCode() , object.GetType() , object.ReferenceEquals(object, object) , object.ToString()

Constructors

AsyncLock()

Initializes a new instance of the AsyncLock class.

```
public AsyncLock()
```

Properties

OnRelease

Gets or sets the callback that should be invoked whenever this lock is released.

```
public Action? OnRelease { get; set; }
```

Property Value

Action

Methods

Dispose()

Releases all resources used by the current instance of the AsyncLock class.

```
public void Dispose()
```

LockAsync()

Asynchronously obtains the lock. Dispose the returned IDisposable to release the lock.

```
public Task<IDisposable> LockAsync()
```

Returns

Task<IDisposable>

The IDisposable that will release the lock.

Class AsyncReaderWriterLock

Namespace: SixLabors.ImageSharp.Web.Synchronization

Assembly: SixLabors.ImageSharp.Web.dll

An asynchronous locker that provides read and write locking policies.

This is based on the following blog post: https://devblogs.microsoft.com/pfxteam/building-async-coordination-primitives-part-7-asyncreaderwriterlock/

```
public class AsyncReaderWriterLock
```

Inheritance

object ← AsyncReaderWriterLock

Inherited Members

object.Equals(object) , object.Equals(object, object) , object.GetHashCode() , object.GetType() , object.MemberwiseClone() , object.ReferenceEquals(object, object) , object.ToString()

Constructors

AsyncReaderWriterLock()

Initializes a new instance of the AsyncReaderWriterLock class.

```
public AsyncReaderWriterLock()
```

Properties

OnRelease

Gets or sets the callback that should be invoked whenever this lock is released.

```
public Action? OnRelease { get; set; }
```

Property Value

Action

Methods

ReaderLockAsync()

Asynchronously obtains the lock in shared reader mode. Dispose the returned IDisposable to release the lock.

```
public Task<IDisposable> ReaderLockAsync()
```

Returns

Task<IDisposable>

The IDisposable that will release the lock.

WriterLockAsync()

Asynchronously obtains the lock in exclusive writer mode. Dispose the returned IDisposable to release the lock.

```
public Task<IDisposable> WriterLockAsync()
```

Returns

Task<IDisposable>

The IDisposable that will release the lock.

Class RefCountedConcurrentDictionary<TKey, TValue>

Namespace: SixLabors.ImageSharp.Web.Synchronization

Assembly: SixLabors.ImageSharp.Web.dll

Represents a thread-safe collection of reference-counted key/value pairs that can be accessed by multiple threads concurrently. Values that don't yet exist are automatically created using a caller supplied value factory method, and when their final refcount is released they are removed from the dictionary.

```
public class RefCountedConcurrentDictionary<TKey, TValue> where TKey : notnull where TValue
: class
```

Type Parameters

`TKey`

The type of the key.

`TValue`

The value for the dictionary.

Inheritance

object ← RefCountedConcurrentDictionary<TKey, TValue>

Inherited Members

object.Equals(object) , object.Equals(object, object) , object.GetHashCode() , object.GetType() , object.MemberwiseClone() , object.ReferenceEquals(object, object) , object.ToString()

Constructors

RefCountedConcurrentDictionary(IEqualityComparer<TKey>, Func<TKey, TValue>, Action<TValue>?)

Initializes a new instance of the RefCountedConcurrentDictionary<TKey, TValue> class that is empty, has the default concurrency level and capacity,, and uses the specified IEqualityComparer<T>.

```
public RefCountedConcurrentDictionary(IEqualityComparer<TKey> comparer, Func<TKey, TValue>
valueFactory, Action<TValue>? valueReleaser)
```

Parameters

`comparer` IEqualityComparer<TKey>

The IEqualityComparer<T> implementation to use when comparing keys.

`valueFactory` Func<TKey, TValue>

Factory method that generates a new `TValue` for a given `TKey`.

`valueReleaser` Action<TValue>

Optional callback that is used to cleanup `TValue`s after their final ref count is released.

RefCountedConcurrentDictionary(Func<TKey, TValue>, Action<TValue>?)

Initializes a new instance of the RefCountedConcurrentDictionary<TKey, TValue> class that is empty, has the default concurrency level, has the default initial capacity, and uses the default comparer for the key type.

```
public RefCountedConcurrentDictionary(Func<TKey, TValue> valueFactory, Action<TValue>?
valueReleaser = null)
```

Parameters

`valueFactory` Func<TKey, TValue>

Factory method that generates a new `TValue` for a given `TKey`.

`valueReleaser` Action<TValue>

Optional callback that is used to cleanup `TValue`s after their final ref count is released.

RefCountedConcurrentDictionary(int, int, IEqualityComparer<TKey>, Func<TKey, TValue>, Action<TValue>?)

Initializes a new instance of the RefCountedConcurrentDictionary<TKey, TValue> class that is empty, has the specified concurrency level, has the specified initial capacity, and uses the specified IEqualityComparer<T>.

```
public RefCountedConcurrentDictionary(int concurrencyLevel, int capacity,
IEqualityComparer<TKey> comparer, Func<TKey, TValue> valueFactory,
Action<TValue>? valueReleaser)
```

Parameters

`concurrencyLevel` int

The estimated number of threads that will access the RefCountedConcurrentDictionary<TKey, TValue> concurrently

`capacity` int

The initial number of elements that the RefCountedConcurrentDictionary<TKey, TValue> can contain.

`comparer` IEqualityComparer<TKey>

The IEqualityComparer<T> implementation to use when comparing keys.

`valueFactory` Func<TKey, TValue>

Factory method that generates a new `TValue` for a given `TKey`.

`valueReleaser` Action<TValue>

Optional callback that is used to cleanup `TValue`s after their final ref count is released.

RefCountedConcurrentDictionary(int, int, Func<TKey, TValue>, Action<TValue>?)

Initializes a new instance of the RefCountedConcurrentDictionary<TKey, TValue> class that is empty, has the specified concurrency level and capacity, and uses the default comparer for the

key type.

```
public RefCountedConcurrentDictionary(int concurrencyLevel, int capacity, Func<TKey, TValue>
valueFactory, Action<TValue>? valueReleaser = null)
```

Parameters

`concurrencyLevel` int

The estimated number of threads that will access the RefCountedConcurrentDictionary<TKey, TValue> concurrently

`capacity` int

The initial number of elements that the RefCountedConcurrentDictionary<TKey, TValue> can contain.

`valueFactory` Func<TKey, TValue>

Factory method that generates a new `TValue` for a given `TKey`.

`valueReleaser` Action<TValue>

Optional callback that is used to cleanup `TValue`s after their final ref count is released.

Methods

Get(TKey)

Obtains a reference to the value corresponding to the specified key. If no such value exists in the dictionary, then a new value is generated using the value factory method supplied in the constructor. To prevent leaks, this reference MUST be released via Release(TKey).

```
public TValue Get(TKey key)
```

Parameters

`key` TKey

The key of the element to add ref.

Returns

TValue

The referenced object.

Release(TKey)

Releases a reference to the value corresponding to the specified key. If this reference was the last remaining reference to the value, then the value is removed from the dictionary, and the optional value releaser callback is invoked.

```
public void Release(TKey key)
```

Parameters

`key` TKey

THe key of the element to release.

Namespace SixLabors.ImageSharp.Web.TagHelpers

Classes

HmacTokenTagHelper

A TagHelper implementation targeting <img> element that allows the automatic generation of HMAC image processing protection tokens.

ImageTagHelper

A TagHelper implementation targeting <img> element that allows the automatic generation image processing commands.

Class HmacTokenTagHelper

Namespace: SixLabors.ImageSharp.Web.TagHelpers

Assembly: SixLabors.ImageSharp.Web.dll

A TagHelper implementation targeting <img> element that allows the automatic generation of HMAC image processing protection tokens.

```
[HtmlTargetElement("img", Attributes = "src", TagStructure = TagStructure.WithoutEndTag)]
public class HmacTokenTagHelper : UrlResolutionTagHelper, ITagHelper, ITagHelperComponent
```

Inheritance

object ← TagHelper ← UrlResolutionTagHelper ← HmacTokenTagHelper

Implements

ITagHelper, ITagHelperComponent

Inherited Members

UrlResolutionTagHelper.ProcessUrlAttribute(string, TagHelperOutput) ,
UrlResolutionTagHelper.TryResolveUrl(string, out string) ,
UrlResolutionTagHelper.TryResolveUrl(string, out IHtmlContent) ,
UrlResolutionTagHelper.UrlHelperFactory , UrlResolutionTagHelper.HtmlEncoder ,
UrlResolutionTagHelper.ViewContext , TagHelper.Init(TagHelperContext) ,
TagHelper.ProcessAsync(TagHelperContext, TagHelperOutput) , object.Equals(object) ,
object.Equals(object, object) , object.GetHashCode() , object.GetType() ,
object.MemberwiseClone() , object.ReferenceEquals(object, object) , object.ToString()

Constructors

HmacTokenTagHelper(IOptions<ImageSharpMiddlewareOptions>, RequestAuthorizationUtilities, IUrlHelperFactory, HtmlEncoder)

Initializes a new instance of the HmacTokenTagHelper class.

```
public HmacTokenTagHelper(IOptions<ImageSharpMiddlewareOptions> options,
RequestAuthorizationUtilities authorizationUtilities, IUrlHelperFactory urlHelperFactory,
```

```
    HtmlEncoder htmlEncoder)
```

Parameters

`options` IOptions<ImageSharpMiddlewareOptions>

The middleware configuration options.

`authorizationUtilities` RequestAuthorizationUtilities

Contains helpers that allow authorization of image requests.

`urlHelperFactory` IUrlHelperFactory

The URL helper factory.

`htmlEncoder` HtmlEncoder

The HTML encoder.

Properties

Order

When a set of ITagHelper's are executed, their Init(TagHelperContext)'s are first invoked in the specified Order; then their ProcessAsync(TagHelperContext, TagHelperOutput)'s are invoked in the specified Order. Lower values are executed first.

```
public override int Order { get; }
```

Property Value

int

Remarks

Default order is `0`.

Src

Gets or sets the source of the image.

```
[HtmlAttributeName("src")]
public string? Src { get; set; }
```

Property Value

string

Remarks

Passed through to the generated HTML in all cases.

Methods

Process(TagHelperContext, TagHelperOutput)

Synchronously executes the TagHelper with the given `context` and `output`.

```
public override void Process(TagHelperContext context, TagHelperOutput output)
```

Parameters

`context` TagHelperContext

Contains information associated with the current HTML tag.

`output` TagHelperOutput

A stateful HTML element used to generate an HTML tag.

Class ImageTagHelper

Namespace: SixLabors.ImageSharp.Web.TagHelpers

Assembly: SixLabors.ImageSharp.Web.dll

A TagHelper implementation targeting <img> element that allows the automatic generation image processing commands.

```
[HtmlTargetElement("img", Attributes = "src,imagesharp-width", TagStructure
= TagStructure.WithoutEndTag)]
[HtmlTargetElement("img", Attributes = "src,imagesharp-height", TagStructure
= TagStructure.WithoutEndTag)]
[HtmlTargetElement("img", Attributes = "src,imagesharp-ranchor", TagStructure
= TagStructure.WithoutEndTag)]
[HtmlTargetElement("img", Attributes = "src,imagesharp-rmode", TagStructure
= TagStructure.WithoutEndTag)]
[HtmlTargetElement("img", Attributes = "src,imagesharp-rxy", TagStructure
= TagStructure.WithoutEndTag)]
[HtmlTargetElement("img", Attributes = "src,imagesharp-rcolor", TagStructure
= TagStructure.WithoutEndTag)]
[HtmlTargetElement("img", Attributes = "src,imagesharp-compand", TagStructure
= TagStructure.WithoutEndTag)]
[HtmlTargetElement("img", Attributes = "src,imagesharp-orient", TagStructure
= TagStructure.WithoutEndTag)]
[HtmlTargetElement("img", Attributes = "src,imagesharp-autoorient", TagStructure
= TagStructure.WithoutEndTag)]
[HtmlTargetElement("img", Attributes = "src,imagesharp-format", TagStructure
= TagStructure.WithoutEndTag)]
[HtmlTargetElement("img", Attributes = "src,imagesharp-bgcolor", TagStructure
= TagStructure.WithoutEndTag)]
[HtmlTargetElement("img", Attributes = "src,imagesharp-quality", TagStructure
= TagStructure.WithoutEndTag)]
public class ImageTagHelper : UrlResolutionTagHelper, ITagHelper, ITagHelperComponent
```

Inheritance

object ← TagHelper ← UrlResolutionTagHelper ← ImageTagHelper

Implements

ITagHelper, ITagHelperComponent

Inherited Members

UrlResolutionTagHelper.ProcessUrlAttribute(string, TagHelperOutput) ,
UrlResolutionTagHelper.TryResolveUrl(string, out string) ,
UrlResolutionTagHelper.TryResolveUrl(string, out IHtmlContent) ,
UrlResolutionTagHelper.UrlHelperFactory , UrlResolutionTagHelper.HtmlEncoder ,
UrlResolutionTagHelper.ViewContext , TagHelper.Init(TagHelperContext) ,
TagHelper.ProcessAsync(TagHelperContext, TagHelperOutput) , object.Equals(object) ,
object.Equals(object, object) , object.GetHashCode() , object.GetType() ,
object.MemberwiseClone() , object.ReferenceEquals(object, object) , object.ToString()

Constructors

ImageTagHelper(IOptions<ImageSharpMiddlewareOptions> , RequestAuthorizationUtilities, IUrlHelperFactory, HtmlEncoder)

Initializes a new instance of the ImageTagHelper class.

```
public ImageTagHelper(IOptions<ImageSharpMiddlewareOptions> options,
RequestAuthorizationUtilities authorizationUtilities, IUrlHelperFactory urlHelperFactory,
HtmlEncoder htmlEncoder)
```

Parameters

`options` IOptions<ImageSharpMiddlewareOptions>

The middleware configuration options.

`authorizationUtilities` RequestAuthorizationUtilities

Contains helpers that allow authorization of image requests.

`urlHelperFactory` IUrlHelperFactory

The URL helper factory.

`htmlEncoder` HtmlEncoder

The HTML encorder.

Properties

AnchorPosition

Gets or sets the anchor position.

```
[HtmlAttributeName("imagesharp-ranchor")]
public AnchorPositionMode? AnchorPosition { get; set; }
```

Property Value

AnchorPositionMode?

AutoOrient

Gets or sets a value indicating whether to automatically rotate/flip the input image based on embedded EXIF orientation property values before processing.

```
[HtmlAttributeName("imagesharp-autoorient")]
public bool? AutoOrient { get; set; }
```

Property Value

bool?

BackgroundColor

Gets or sets the background color of the image.

```
[HtmlAttributeName("imagesharp-bgcolor")]
public Color? BackgroundColor { get; set; }
```

Property Value

Color?

Center

Gets or sets the center coordinates.

```
[HtmlAttributeName("imagesharp-rxy")]
public PointF? Center { get; set; }
```

Property Value

PointF?

Compand

Gets or sets a value indicating whether to compress or expand individual pixel colors values on processing.

```
[HtmlAttributeName("imagesharp-compand")]
public bool? Compand { get; set; }
```

Property Value

bool?

Format

Gets or sets the image format to convert to.

```
[HtmlAttributeName("imagesharp-format")]
public FormatCommand? Format { get; set; }
```

Property Value

FormatCommand?

Height

Gets or sets the height in pixel units.

```
[HtmlAttributeName("imagesharp-height")]
public int? Height { get; set; }
```

Property Value

int?

Remarks

Passed through to the generated HTML in all cases.

Order

When a set of ITagHelper's are executed, their Init(TagHelperContext)'s are first invoked in the specified Order; then their ProcessAsync(TagHelperContext, TagHelperOutput)'s are invoked in the specified Order. Lower values are executed first.

```
public override int Order { get; }
```

Property Value

int

Remarks

Default order is `0`.

Orient

Gets or sets a value indicating whether to factor embedded EXIF orientation property values during processing.

```
[HtmlAttributeName("imagesharp-orient")]
public bool? Orient { get; set; }
```

Property Value

bool?

Remarks

Defaults to true.

PadColor

Gets or sets the color to use as a background when padding an image.

```
[HtmlAttributeName("imagesharp-rcolor")]
public Color? PadColor { get; set; }
```

Property Value

Color?

Quality

Gets or sets the quality, that will be used to encode the image. Quality index must be between 0 and 100 (compression from max to min).

```
[HtmlAttributeName("imagesharp-quality")]
public int? Quality { get; set; }
```

Property Value

int?

ResizeMode

Gets or sets the resize mode.

```
[HtmlAttributeName("imagesharp-rmode")]
```

```
public ResizeMode? ResizeMode { get; set; }
```

Property Value

ResizeMode?

Sampler

Gets or sets the sampling algorithm to use when resizing images.

```
[HtmlAttributeName("imagesharp-rsampler")]
public ResamplerCommand? Sampler { get; set; }
```

Property Value

ResamplerCommand?

Src

Gets or sets the src.

```
[HtmlAttributeName("src")]
public string? Src { get; set; }
```

Property Value

string

Remarks

Passed through to the generated HTML in all cases.

Width

Gets or sets the width in pixel units.

```
[HtmlAttributeName("imagesharp-width")]
public int? Width { get; set; }
```

Property Value

int?

Remarks

Passed through to the generated HTML in all cases.

Methods

AddProcessingCommands(TagHelperContext, TagHelperOutput, CommandCollection, CultureInfo)

Allows the addition of processing commands by inheriting classes.

```
protected virtual void AddProcessingCommands(TagHelperContext context, TagHelperOutput
output, CommandCollection commands, CultureInfo commandCulture)
```

Parameters

`context` TagHelperContext

Contains information associated with the current HTML tag.

`output` TagHelperOutput

A stateful HTML element used to generate an HTML tag.

`commands` CommandCollection

The command collection.

`commandCulture` CultureInfo

The culture to use when generating and processing commands.

Process(TagHelperContext, TagHelperOutput)

Synchronously executes the TagHelper with the given `context` and `output`.

```
public override void Process(TagHelperContext context, TagHelperOutput output)
```

Parameters

`context` TagHelperContext

Contains information associated with the current HTML tag.

`output` TagHelperOutput

A stateful HTML element used to generate an HTML tag.